

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC
ĐỀ TÀI: Docker CIS Benchmark

Môn học: NT542.Q22 - Lập trình kịch bản tự động hóa cho quản trị và bảo mật mạng

Giảng viên hướng dẫn: Th.S Văn Thiên Luân

Thực hiện bởi nhóm 01, bao gồm:

1. Đỗ Gia Hân	23520425	Trưởng nhóm
2. Bùi Nguyễn Tường Vân	23521768	Thành viên
3. Vũ Duy An	23520034	Thành viên
4. Phan Nguyễn Hoài An	23520034	Thành viên

Thời gian thực hiện: 01/04/2026 - 31/05/2026

MỤC LỤC

MỤC LỤC	2
Chương I. TỔNG QUAN BENCHMARK.....	4
1. Bối cảnh thực tế	4
1.1. Về CIS và Mục tiêu của CIS Benchmarks.....	4
1.2. Mục tiêu Benchmark GCP Foundation v1.8.0.....	4
1.3. Đối tượng sử dụng Benchmark.....	4
1.4. Quy trình xây dựng dựa trên sự đồng thuận	5
1.5. Tổng quan Docker	5
2. Nội dung Benchmark.....	7
2.1. Host Configuration	8
2.2. Docker Daemon Configuration.....	8
2.3. Docker Daemon Configuration Files.....	9
2.4. Container Images and Build File Configuration.....	9
2.5. Container Runtime Configuration	9
2.6. Docker Security Operations.....	10
2.7. Docker Swarm Configuration.....	10
3. Mức độ đánh giá.	11
3.1. Level 1 - Docker - Linux	11
3.2. Level 2 - Docker - Linux	11
3.3. Level 1 - Docker Swarm.....	12
Chương II. THIẾT KẾ HỆ THỐNG.....	13
1. Kiến trúc ứng dụng	13
1.1. Sơ đồ kiến trúc hệ thống (giải thích công cụ sử dụng).....	13
1.2. Luồng hoạt động hệ thống	13
2. Phương pháp thực hiện Manual.....	14
2.1. Manual Audit	14
2.2. Manual Remediation.....	16
3. Phương pháp thực hiện Automation	16
3.1. Mục tiêu	16

3.2. Luồng hoạt động Automation	17
Chương III. TRIỂN KHAI HỆ THỐNG	18
1. Triển khai hạ tầng	18
1.1. Cài đặt Docker và Docker Compos	18
1.2. Cấu hình Security Group	18
1.3. Triển khai Docker Compose	18
1.4. Quản lý biến môi trường.....	19
1.5. Thiết lập GitHub Runner	20
1.6. Kiểm tra hoạt động hệ thống.....	21
2. Triển khai Manual.....	22
2.1. Module 1:.....	22
2.2. Module 2:.....	30
1.1.8 Ensure auditing is configured for Docker files and directories- containerd.sock	32
2.3. Module 3:.....	44
2.4. Module 4:.....	54
3. Triển khai Automation	67
3.1. Cấu hình GitHub Secrets	68
3.2. Workflow CI/CD trên GitHub Actions	68
3.3. Thiết kế script CIS Audit	69
3.4. Liên kết EC2 với GitHub Actions (kèm cơ chế API & authentication).....	69
Chương IV. DEMO	71
NGUỒN THAM KHẢO.....	78

Chương I. TỔNG QUAN BENCHMARK

1. Bối cảnh thực tế

1.1. Về CIS và Mục tiêu của CIS Benchmarks

Center for Internet Security (CIS) là một tổ chức phi lợi nhuận chuyên nghiên cứu và phát triển các tiêu chuẩn bảo mật dành cho hệ thống công nghệ thông tin. CIS được thành lập với mục tiêu nâng cao mức độ an toàn cho các hệ thống mạng, máy chủ, nền tảng điện toán đám mây và ứng dụng thông qua việc xây dựng các bộ hướng dẫn cấu hình bảo mật tiêu chuẩn. Các tài liệu do CIS phát hành được sử dụng rộng rãi trong doanh nghiệp, tổ chức chính phủ và môi trường học thuật nhằm giảm thiểu các rủi ro bảo mật phổ biến.

Một trong những sản phẩm quan trọng nhất của CIS là CIS Benchmarks. Đây là tập hợp các khuyến nghị bảo mật được xây dựng dựa trên kinh nghiệm thực tế của các chuyên gia an ninh mạng trên toàn thế giới. Mỗi benchmark cung cấp các cấu hình bảo mật cụ thể nhằm giúp hệ thống vận hành an toàn hơn, hạn chế sai sót cấu hình và giảm bề mặt tấn công. Các khuyến nghị trong CIS Benchmarks thường tập trung vào việc tăng cường quyền kiểm soát truy cập, hạn chế dịch vụ không cần thiết, bảo vệ dữ liệu và nâng cao khả năng giám sát hệ thống.

CIS Benchmarks không chỉ đóng vai trò là tài liệu hướng dẫn kỹ thuật mà còn là cơ sở để doanh nghiệp xây dựng chính sách bảo mật nội bộ, phục vụ kiểm thử an toàn thông tin và đáp ứng các yêu cầu tuân thủ trong nhiều tiêu chuẩn bảo mật quốc tế.

1.2. Mục tiêu Benchmark GCP Foundation v1.8.0

Benchmark GCP Foundation v1.8.0 được xây dựng nhằm cung cấp các hướng dẫn bảo mật dành cho môi trường Google Cloud Platform (GCP). Bộ benchmark này tập trung vào việc cấu hình an toàn các dịch vụ cốt lõi của GCP như Identity and Access Management (IAM), logging, monitoring, networking, storage và compute resources.

Mục tiêu chính của benchmark là giúp tổ chức giảm thiểu các nguy cơ phát sinh từ cấu hình sai hoặc quản trị chưa chặt chẽ trong môi trường điện toán đám mây. Thông qua việc áp dụng các khuyến nghị của CIS, doanh nghiệp có thể tăng cường khả năng bảo vệ tài nguyên cloud, hạn chế truy cập trái phép và cải thiện khả năng phát hiện sự cố bảo mật. Ngoài ra, benchmark còn giúp chuẩn hóa quy trình vận hành hệ thống trên nền tảng GCP. Việc sử dụng các cấu hình bảo mật thống nhất giúp giảm thiểu sự khác biệt giữa các môi trường triển khai, đồng thời hỗ trợ doanh nghiệp đáp ứng các yêu cầu kiểm toán và tuân thủ an toàn thông tin.

1.3. Đối tượng sử dụng Benchmark

CIS Benchmarks được thiết kế để phục vụ nhiều nhóm đối tượng khác nhau trong lĩnh vực công nghệ thông tin và an toàn thông tin. Đối tượng sử dụng chính bao gồm quản

trị viên hệ thống, kỹ sư bảo mật, chuyên gia DevOps, quản trị cloud và các tổ chức triển khai hạ tầng CNTT.

Đối với quản trị viên hệ thống, benchmark cung cấp các hướng dẫn cấu hình cụ thể giúp tăng cường mức độ an toàn cho máy chủ và dịch vụ đang vận hành. Đối với chuyên gia bảo mật, benchmark đóng vai trò là tài liệu tham chiếu để đánh giá mức độ tuân thủ và phát hiện các điểm yếu bảo mật trong hệ thống. Ngoài ra, các doanh nghiệp triển khai mô hình DevOps hoặc Cloud Computing cũng sử dụng CIS Benchmarks như một chuẩn bảo mật cơ sở trong quá trình xây dựng và triển khai hạ tầng. Điều này giúp đảm bảo rằng các dịch vụ được cấu hình theo hướng an toàn ngay từ đầu, giảm nguy cơ phát sinh lỗi hỏng trong quá trình vận hành thực tế.

1.4. Quy trình xây dựng dựa trên sự đồng thuận

Một đặc điểm quan trọng của CIS Benchmarks là các tài liệu này được xây dựng dựa trên mô hình đồng thuận cộng đồng. Quá trình phát triển benchmark có sự tham gia của nhiều chuyên gia đến từ các tổ chức công nghệ, doanh nghiệp bảo mật, cơ quan chính phủ và cộng đồng nghiên cứu an toàn thông tin.

Trong quá trình xây dựng, các khuyến nghị bảo mật sẽ được đề xuất, thảo luận, đánh giá và phản biện bởi nhiều chuyên gia khác nhau. Điều này giúp đảm bảo rằng các cấu hình được đưa ra có tính thực tiễn cao, phù hợp với nhiều môi trường triển khai và không gây ảnh hưởng tiêu cực đến hoạt động của hệ thống.

Cơ chế đồng thuận còn giúp benchmark luôn được cập nhật theo sự thay đổi của công nghệ và các mối đe dọa an ninh mạng mới. Nhờ đó, CIS Benchmarks trở thành một trong những bộ tiêu chuẩn bảo mật được tin cậy và sử dụng rộng rãi trên thế giới.

1.5. Tổng quan Docker



1.5.1. Docker là gì

Docker là một nền tảng mã nguồn mở hỗ trợ phát triển, triển khai và vận hành ứng dụng thông qua công nghệ container. Thay vì cài đặt trực tiếp ứng dụng lên hệ điều hành vật lý hoặc máy ảo, Docker cho phép đóng gói ứng dụng cùng toàn bộ thư viện và môi trường cần thiết vào một container độc lập.

Container hoạt động như một môi trường thực thi nhẹ, có khả năng chạy ổn định trên nhiều hệ thống khác nhau. Điều này giúp giải quyết vấn đề khác biệt môi trường giữa quá trình phát triển, kiểm thử và triển khai thực tế. Docker giúp tăng tính linh hoạt trong triển khai ứng dụng, giảm tài nguyên tiêu thụ và rút ngắn thời gian phát triển phần mềm.

Hiện nay Docker được sử dụng rộng rãi trong các hệ thống microservices, DevOps, CI/CD và điện toán đám mây nhờ khả năng triển khai nhanh và quản lý ứng dụng hiệu quả.

1.5.2. Kiến trúc Docker

Kiến trúc Docker được xây dựng theo mô hình client-server. Thành phần chính bao gồm Docker Client, Docker Daemon và Docker Registry. Docker Client là nơi người dùng thực hiện các lệnh quản lý container như build image, chạy container hoặc kiểm tra trạng thái hệ thống. Docker Daemon đóng vai trò xử lý các yêu cầu từ client và quản lý toàn bộ tài nguyên Docker trên máy chủ. Thành phần này chịu trách nhiệm tạo image, khởi chạy container, quản lý network và storage.

Ngoài ra, Docker Registry là nơi lưu trữ và phân phối Docker image. Người dùng có thể tải image từ Docker Hub hoặc xây dựng registry riêng để phục vụ hệ thống nội bộ. Nhờ kiến trúc này, Docker cho phép quản lý container một cách linh hoạt và dễ dàng mở rộng trong môi trường doanh nghiệp.

1.5.3. 1.5.3. Docker Engine

Docker Engine là thành phần cốt lõi của nền tảng Docker, chịu trách nhiệm tạo và quản lý container. Docker Engine bao gồm Docker Daemon, REST API và Docker CLI.

Docker Engine cung cấp môi trường thực thi cho container, đồng thời quản lý image, network và storage liên quan đến container đó. Thông qua Docker Engine, người dùng có thể triển khai ứng dụng nhanh chóng chỉ với vài lệnh đơn giản.

Một ưu điểm quan trọng của Docker Engine là khả năng hoạt động nhất quán trên nhiều hệ điều hành khác nhau như Linux, Windows và macOS. Điều này giúp ứng dụng được triển khai đồng nhất trong nhiều môi trường khác nhau.

1.5.4. 1.5.4. Docker Engine

Docker daemon là tiến trình nền chịu trách nhiệm quản lý các đối tượng Docker như image, container, network và volume. Thành phần này thường được chạy dưới tên tiến trình dockerd.

Khi người dùng thực hiện lệnh Docker từ command line, yêu cầu sẽ được gửi tới Docker daemon thông qua REST API. Docker daemon sau đó sẽ xử lý yêu cầu tương ứng như tạo container mới, tải image hoặc khởi động dịch vụ.

Do Docker daemon có quyền truy cập sâu vào hệ thống host nên đây là thành phần rất quan trọng về mặt bảo mật. Nếu daemon bị khai thác hoặc cấu hình không an toàn, kẻ tấn công có thể kiểm soát toàn bộ hệ thống chạy Docker.

1.5.5. Container runtime

Container runtime là thành phần chịu trách nhiệm thực thi container trên hệ điều hành host. Runtime thực hiện việc tạo tiến trình container, thiết lập namespace, cgroup và cô lập tài nguyên hệ thống giữa các container với nhau.

Trong Docker, container runtime mặc định thường sử dụng containerd và runc. Containerd chịu trách nhiệm quản lý vòng đời container, trong khi runc trực tiếp tạo và chạy container theo chuẩn OCI (Open Container Initiative). Việc sử dụng container runtime giúp container hoạt động độc lập với hệ điều hành host, đồng thời đảm bảo tính linh hoạt và khả năng tương thích giữa nhiều nền tảng container khác nhau.

1.5.6. Docker networking

Docker networking là cơ chế cho phép container giao tiếp với nhau, với host và với mạng bên ngoài. Docker hỗ trợ nhiều mô hình network khác nhau như bridge network, host network, overlay network và macvlan network. Mặc định, Docker thường sử dụng bridge network để kết nối các container trên cùng một host. Trong mô hình này, mỗi container được cấp phát địa chỉ IP riêng và có thể giao tiếp thông qua virtual bridge.

Ngoài việc hỗ trợ kết nối mạng nội bộ, Docker networking còn cho phép publish port từ container ra bên ngoài để người dùng hoặc dịch vụ khác có thể truy cập ứng dụng đang chạy trong container. Do networking liên quan trực tiếp đến giao tiếp dữ liệu nên đây cũng là một yếu tố quan trọng trong bảo mật Docker. Việc cấu hình sai network có thể tạo điều kiện cho truy cập trái phép hoặc tấn công giữa các container.

1.5.7. Storage driver

Storage driver là thành phần chịu trách nhiệm quản lý dữ liệu và filesystem của Docker image cũng như container. Docker sử dụng cơ chế layered filesystem, trong đó mỗi image được chia thành nhiều lớp dữ liệu khác nhau nhằm tối ưu dung lượng lưu trữ và tốc độ triển khai.

Một số storage driver phổ biến trong Docker bao gồm overlay2, aufs, devicemapper và btrfs. Trong đó, overlay2 hiện là storage driver được khuyến nghị sử dụng rộng rãi trên Linux do có hiệu năng tốt và khả năng tương thích cao. Storage driver ảnh hưởng trực tiếp đến hiệu năng đọc ghi dữ liệu của container. Ngoài ra, việc lựa chọn storage driver phù hợp còn giúp cải thiện khả năng ổn định và bảo mật của hệ thống Docker.

2. Nội dung Benchmark

CIS Docker Benchmark cung cấp tập hợp các khuyến nghị bảo mật dành cho môi trường Docker nhằm giúp quản trị viên và tổ chức triển khai container theo hướng an toàn hơn. Các khuyến nghị này tập trung vào nhiều thành phần khác nhau của hệ thống Docker, từ máy chủ host, Docker daemon, image, container runtime cho đến các hoạt động vận hành thực tế.

Benchmark được chia thành nhiều nhóm nội dung nhằm hỗ trợ việc kiểm tra, đánh giá và triển khai bảo mật một cách có hệ thống. Mỗi nhóm sẽ tập trung vào một lớp thành phần cụ thể trong kiến trúc Docker và đưa ra các yêu cầu cấu hình tương ứng.

2.1. Host Configuration

Host Configuration tập trung vào việc bảo mật hệ điều hành host dùng để chạy Docker. Đây là lớp nền tảng quan trọng vì toàn bộ container đều hoạt động dựa trên tài nguyên của host. Nếu hệ điều hành host bị khai thác hoặc cấu hình không an toàn thì toàn bộ container và dữ liệu liên quan đều có nguy cơ bị ảnh hưởng.

Các khuyến nghị trong nhóm này chủ yếu liên quan đến việc cấu hình filesystem, phân quyền người dùng, auditing và hardening hệ điều hành. Benchmark yêu cầu tách riêng thư mục `/var/lib/docker` sang phân vùng độc lập nhằm hạn chế nguy cơ làm đầy ổ đĩa hệ thống và hỗ trợ cô lập dữ liệu container. Ngoài ra, benchmark cũng yêu cầu kiểm soát chặt chẽ các tài khoản thuộc nhóm *docker* do những tài khoản này gần tương đương với quyền root trên host.

Bên cạnh đó, Host Configuration còn bao gồm việc thiết lập audit log cho Docker daemon và các file quan trọng nhằm tăng khả năng theo dõi hoạt động hệ thống và phát hiện các hành vi bất thường. Các cấu hình liên quan đến firewall, SSH và các dịch vụ không cần thiết trên host cũng đóng vai trò quan trọng trong việc giảm bề mặt tấn công đối với hệ thống Docker.

2.2. Docker Daemon Configuration

Docker Daemon Configuration tập trung vào việc cấu hình bảo mật cho Docker daemon, thành phần trung tâm chịu trách nhiệm quản lý container và tài nguyên Docker trên hệ thống. Do Docker daemon có quyền truy cập sâu vào hệ điều hành host nên việc cấu hình không an toàn có thể dẫn đến nguy cơ bị kiểm soát toàn bộ hệ thống.

Các khuyến nghị trong nhóm này bao gồm việc chạy Docker daemon ở chế độ rootless nếu có thể nhằm giảm quyền đặc biệt của tiến trình Docker. Benchmark cũng khuyến nghị hạn chế giao tiếp giữa các container trên bridge mặc định để tránh nguy cơ tấn công ngang hàng giữa container với nhau.

Ngoài ra, benchmark yêu cầu cấu hình mức log phù hợp, bật iptables để Docker có thể quản lý lưu lượng mạng an toàn, sử dụng storage driver được hỗ trợ và tránh sử dụng các driver đã lỗi thời như *aufs* hoặc *devicemapper*.

Nhóm cấu hình này còn liên quan đến các cơ chế bảo mật nâng cao như TLS cho Docker daemon, user namespace remapping, seccomp profile, cgroup, live restore và hạn chế sử dụng các tính năng experimental. Các khuyến nghị này giúp tăng khả năng cô lập container, giảm nguy cơ leo thang đặc quyền và nâng cao tính ổn định của hệ thống.

2.3. Docker Daemon Configuration Files

Docker Daemon Configuration Files tập trung vào việc bảo vệ các file và thư mục cấu hình quan trọng liên quan đến Docker daemon. Những file này chứa thông tin cấu hình hệ thống, socket giao tiếp và các tham số ảnh hưởng trực tiếp đến hoạt động của Docker.

Benchmark yêu cầu các file như *docker.service*, *docker.socket*, *daemon.json* và thư mục */etc/docker* phải có quyền sở hữu thuộc về *root* nhằm ngăn người dùng thông thường thay đổi cấu hình Docker trái phép. Đồng thời, các file này cũng phải được thiết lập permission phù hợp để hạn chế quyền ghi từ các tài khoản không được ủy quyền.

Ngoài ra, benchmark cũng kiểm tra quyền truy cập của file *docker.sock*, đây là socket giao tiếp trực tiếp với Docker daemon. Nếu socket này bị cấp quyền quá rộng, người dùng trái phép có thể điều khiển Docker daemon và từ đó kiểm soát toàn bộ hệ thống host.

Bên cạnh Docker daemon, benchmark còn đề cập đến các file cấu hình và socket liên quan đến *containerd*, do đây là runtime thành phần quan trọng trong kiến trúc Docker hiện đại.

2.4. Container Images and Build File Configuration

Container Images and Build File Configuration tập trung vào việc tăng cường bảo mật cho Docker image và Dockerfile. Image là thành phần nền tảng dùng để tạo container, vì vậy các lỗ hổng hoặc cấu hình không an toàn trong image có thể ảnh hưởng trực tiếp đến toàn bộ ứng dụng được triển khai.

Benchmark khuyến nghị chỉ sử dụng các image nền đáng tin cậy, có nguồn gốc rõ ràng và được cập nhật thường xuyên nhằm giảm nguy cơ chứa mã độc hoặc lỗ hổng bảo mật. Ngoài ra, image cần được tối ưu để hạn chế cài đặt các package không cần thiết nhằm giảm bề mặt tấn công.

Một số khuyến nghị khác bao gồm việc tạo user riêng bên trong container thay vì chạy ứng dụng bằng tài khoản root, sử dụng lệnh *COPY* thay cho *ADD* trong Dockerfile để hạn chế hành vi tự động giải nén không mong muốn, và bổ sung chỉ dẫn *HEALTHCHECK* nhằm hỗ trợ kiểm tra trạng thái hoạt động của ứng dụng.

Benchmark cũng khuyến khích sử dụng cơ chế scan image và Docker Content Trust để đảm bảo tính toàn vẹn của image trước khi triển khai vào môi trường thực tế.

2.5. Container Runtime Configuration

Container Runtime Configuration tập trung vào việc kiểm soát cách container được khởi chạy và vận hành trên hệ thống. Đây là nhóm cấu hình rất quan trọng do container runtime là nơi trực tiếp thực thi ứng dụng và tương tác với kernel của hệ điều hành host.

Benchmark yêu cầu hạn chế sử dụng container đặc quyền (*--privileged*) vì loại container này có khả năng truy cập gần như toàn bộ tài nguyên của host. Ngoài ra, benchmark

cũng khuyến nghị giới hạn Linux capabilities, không mount các thư mục nhạy cảm từ host vào container và không chia sẻ namespace của host nhằm giảm nguy cơ leo thang đặc quyền.

Các khuyến nghị khác bao gồm thiết lập giới hạn tài nguyên như CPU, memory và PID để ngăn container sử dụng quá mức tài nguyên hệ thống dẫn đến tấn công từ chối dịch vụ. Benchmark cũng yêu cầu duy trì seccomp profile mặc định, bật AppArmor hoặc SELinux nếu có thể và tránh mount trực tiếp *docker.sock* vào container. Những cấu hình này giúp tăng mức độ cô lập giữa container với host, đồng thời hạn chế tác động nếu container bị khai thác hoặc xâm nhập.

2.6. Docker Security Operations

Docker Security Operations tập trung vào các hoạt động quản trị và vận hành bảo mật trong môi trường Docker thực tế. Không chỉ dừng ở cấu hình hệ thống, benchmark còn nhấn mạnh tầm quan trọng của việc duy trì môi trường Docker gọn gàng, được giám sát và cập nhật thường xuyên.

Một trong những nội dung chính của nhóm này là hạn chế tình trạng image sprawl và container sprawl, tức là tránh tồn tại quá nhiều image hoặc container không còn sử dụng. Việc duy trì quá nhiều image cũ hoặc container không cần thiết có thể làm tăng nguy cơ tồn tại lỗ hổng chưa được vá, gây khó khăn cho quá trình quản lý và tiêu tốn tài nguyên hệ thống.

Ngoài ra, benchmark cũng khuyến khích việc theo dõi log, giám sát hoạt động container và thường xuyên kiểm tra tính tuân thủ bảo mật của môi trường Docker. Việc vận hành Docker theo quy trình bảo mật chuẩn giúp doanh nghiệp nâng cao khả năng phát hiện sự cố và giảm thiểu rủi ro trong quá trình triển khai thực tế.

2.7. Docker Swarm Configuration

Docker Swarm Configuration tập trung vào các khuyến nghị bảo mật dành cho môi trường sử dụng Docker Swarm. Swarm là cơ chế orchestration của Docker cho phép quản lý nhiều node và triển khai container trên quy mô lớn.

Trong môi trường Swarm, benchmark yêu cầu bảo vệ chặt chẽ các manager node vì đây là thành phần điều phối toàn bộ cluster. Nếu manager node bị kiểm soát, toàn bộ cụm Swarm có thể bị ảnh hưởng.

Ngoài ra, benchmark còn khuyến nghị bật mã hóa overlay network nhằm bảo vệ dữ liệu truyền giữa các node trong cluster. Docker Swarm cũng cần được cấu hình quản lý secret an toàn để tránh lưu trữ thông tin nhạy cảm trực tiếp trong image hoặc environment variable.

Một số khuyến nghị khác bao gồm bật auto-lock cho Swarm manager, định kỳ xoay vòng khóa mã hóa và certificate nhằm giảm nguy cơ lộ khóa trong thời gian dài.

Benchmark cũng khuyến nghị tách biệt lưu lượng quản trị với lưu lượng dữ liệu để tăng mức độ an toàn cho hệ thống Docker Swarm.

3. Mức độ đánh giá.

Trong CIS Docker Benchmark, các khuyến nghị bảo mật được phân chia thành nhiều mức độ nhằm hỗ trợ tổ chức lựa chọn cấu hình phù hợp với nhu cầu vận hành thực tế. Việc phân cấp này giúp cân bằng giữa khả năng bảo mật, hiệu năng hệ thống và tính tương thích của ứng dụng.

Các mức đánh giá trong CIS Docker Benchmark bao gồm Level 1 - Docker - Linux, Level 2 - Docker - Linux và Level 1 - Docker Swarm. Mỗi mức sẽ có phạm vi áp dụng và mức độ ảnh hưởng khác nhau đối với môi trường Docker.

3.1. Level 1 - Docker - Linux

Level 1 - Docker - Linux bao gồm các khuyến nghị bảo mật cơ bản nhưng quan trọng dành cho môi trường Docker chạy trên hệ điều hành Linux. Các cấu hình trong nhóm này được thiết kế nhằm giảm thiểu các rủi ro bảo mật phổ biến mà vẫn đảm bảo hệ thống hoạt động ổn định và tương thích với phần lớn ứng dụng thực tế.

Các khuyến nghị thuộc Level 1 thường tập trung vào việc kiểm soát quyền truy cập, phân quyền file cấu hình, cấu hình logging, auditing, quản lý Docker daemon và hạn chế các thiết lập không an toàn phổ biến. Đây là nhóm cấu hình có tác động thấp đến hiệu năng và quy trình vận hành nên được khuyến nghị áp dụng cho hầu hết hệ thống Docker.

Trong phạm vi đồ án, phần lớn các nội dung kiểm tra và triển khai được thực hiện dựa trên các khuyến nghị thuộc Level 1 nhằm đảm bảo tính khả thi và phù hợp với môi trường Docker thực tế.

3.2. Level 2 - Docker - Linux

Level 2 - Docker - Linux bao gồm các khuyến nghị bảo mật nâng cao dành cho môi trường Docker yêu cầu mức độ an toàn cao hơn. So với Level 1, các cấu hình trong nhóm này thường nghiêm ngặt hơn và có thể ảnh hưởng đến hiệu năng, tính linh hoạt hoặc khả năng tương thích của hệ thống.

Các khuyến nghị thuộc Level 2 tập trung vào việc tăng cường cơ chế cô lập container, hạn chế quyền truy cập hệ thống, kiểm soát tài nguyên chặt chẽ và áp dụng các cơ chế hardening nâng cao cho Docker daemon cũng như container runtime. Những cấu hình này giúp giảm thiểu nguy cơ leo thang đặc quyền và tăng khả năng chống chịu trước các cuộc tấn công nhắm vào môi trường container.

Level 2 thường phù hợp với các hệ thống production, môi trường xử lý dữ liệu nhạy cảm hoặc các tổ chức có yêu cầu tuân thủ bảo mật nghiêm ngặt. Tuy nhiên, trước khi triển khai cần đánh giá kỹ ảnh hưởng của các cấu hình này đến ứng dụng và quy trình vận hành hiện tại.

3.3. Level 1 - Docker Swarm

Level 1 - Docker Swarm bao gồm các khuyến nghị bảo mật dành cho môi trường sử dụng Docker Swarm để quản lý cluster container. Nhóm cấu hình này tập trung vào việc bảo vệ các manager node, mã hóa overlay network, quản lý secret, certificate rotation và kiểm soát lưu lượng giữa các node trong cụm Swarm.

Mục tiêu của Level 1 - Docker Swarm là đảm bảo các thành phần orchestration của Docker được triển khai theo hướng an toàn nhưng vẫn duy trì tính ổn định và khả năng mở rộng của hệ thống. Nếu hệ thống không sử dụng Docker Swarm thì các khuyến nghị trong nhóm này có thể được xem là không áp dụng trong quá trình đánh giá.

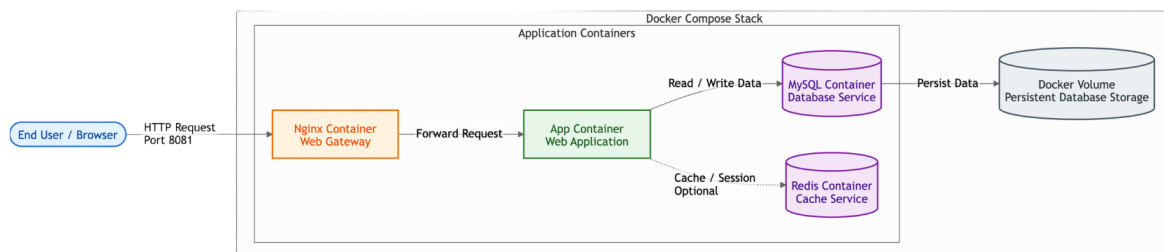
Chương II. THIẾT KẾ HỆ THỐNG

1. Kiến trúc ứng dụng

1.1. Sơ đồ kiến trúc hệ thống (giải thích công cụ sử dụng)

Hệ thống được thiết kế theo mô hình containerized infrastructure sử dụng Docker nhằm triển khai một môi trường web application có khả năng áp dụng các kỹ thuật hardening theo tiêu chuẩn CIS Docker Benchmark.

Toàn bộ hệ thống được triển khai trên nền tảng AWS EC2 Ubuntu Server. Các thành phần ứng dụng được tách thành nhiều container độc lập nhằm tăng khả năng cô lập, mở rộng và quản lý bảo mật.



Hệ thống bao gồm các thành phần chính:

- Nginx container đóng vai trò reverse proxy và web server.
- PHP-FPM container xử lý logic ứng dụng.
- MySQL container lưu trữ dữ liệu.
- Redis container hỗ trợ cache và tăng hiệu năng.
- Docker Compose quản lý multi-container.
- GitHub Actions và Self-hosted Runner hỗ trợ CI/CD automation.
- Bash audit scripts thực hiện kiểm tra CIS Docker Benchmark.

1.2. Luồng hoạt động hệ thống

1.2.1. Luồng truy cập người dùng

Quá trình xử lý request của hệ thống diễn ra như sau:

- 1) Người dùng truy cập Public IP của EC2.
- 2) Request đi qua AWS Security Group.
- 3) Nginx container tiếp nhận request HTTP.
- 4) Nginx forward request sang PHP-FPM container.
- 5) PHP Application xử lý logic nghiệp vụ.
- 6) Ứng dụng truy cập MySQL và Redis thông qua Docker internal network.
- 7) Kết quả xử lý được trả ngược về Nginx.
- 8) Response được gửi lại cho người dùng.

1.2.2. Luồng giao tiếp giữa các container

Các container giao tiếp với nhau thông qua Docker internal network nhằm đảm bảo tính cô lập và hạn chế truy cập trái phép.

- Nginx chỉ giao tiếp với PHP-FPM.
- PHP-FPM được phép truy cập MySQL và Redis.
- Database container không expose trực tiếp ra Internet.
- Redis chỉ cho phép truy cập nội bộ.

Thiết kế này giúp:

- Giảm nguy cơ tấn công trực tiếp vào database
- Tăng tính bảo mật hệ thống
- Hạn chế lateral movement giữa các service

2. Phương pháp thực hiện Manual

2.1. Manual Audit

Trước khi triển khai automation, hệ thống được kiểm tra thủ công nhằm đánh giá mức độ tuân thủ theo CIS Docker Benchmark và xác định các cấu hình chưa an toàn trong Docker Engine, container runtime, image và network configuration.

2.1.1. Mục tiêu

Quá trình Manual Audit được thực hiện nhằm:

- Đánh giá mức độ an toàn của hạ tầng Docker trước khi đưa vào vận hành.
- Xác định các cấu hình chưa tuân thủ CIS Docker Benchmark.
- Kiểm tra mức độ hardening của Docker daemon, container runtime và image.
- Phát hiện các rủi ro bảo mật như chạy container bằng quyền root, thiếu healthcheck, mở port không cần thiết hoặc sử dụng image chưa được hardening.
- Làm cơ sở xây dựng các script automation và remediation ở các bước tiếp theo.

2.1.2. Công cụ sử dụng

Công cụ	Chức năng
docker inspect	Kiểm tra cấu hình chi tiết của container và image
docker info	Kiểm tra Docker daemon và security options
docker ps	Kiểm tra trạng thái container runtime
docker history	Kiểm tra lịch sử build image
stat	Kiểm tra owner và permission của file
auditctl	Kiểm tra audit rules của hệ thống

curl	Kiểm tra khả năng hoạt động của service
ss / netstat	Kiểm tra port đang mở
systemctl	Kiểm tra trạng thái Docker service

2.1.3. Các nội dung audit

– Infrastructure & Hardening

Module này tập trung kiểm tra và hardening lớp hạ tầng nền của Docker Host, bao gồm hệ điều hành, Docker daemon, storage driver, file cấu hình, socket và các thiết lập permission quan trọng.

Mục tiêu là đảm bảo Docker Host được cấu hình an toàn trước khi triển khai container nhằm giảm nguy cơ privilege escalation hoặc attacker kiểm soát Docker daemon.

Các tiêu chí CIS được kiểm tra: 1.1.1, 1.1.2, 1.2.1, 1.2.2, 2.6, 2.7, 2.11, 2.12, 3.1–3.6, 3.15–3.16, 3.23–3.24.

– Audit, Logging & Trust

Module này tập trung vào khả năng giám sát, logging, auditing và bảo vệ các cơ chế xác thực của Docker như TLS, certificate và Docker daemon. Mục tiêu là hỗ trợ truy vết hoạt động hệ thống, phát hiện thay đổi trái phép và tăng độ tin cậy của Docker Host.

Các tiêu chí CIS được kiểm tra: 1.1.3–1.1.18, 2.1–2.5, 2.8, 2.14, 3.7–3.14.

– Image & Supply Chain Security Audit

Module này kiểm tra tính an toàn của Docker Image, Dockerfile và chuỗi cung ứng container nhằm đảm bảo image sạch, đáng tin cậy và không chứa mã độc hoặc secrets. Ngoài ra module còn đánh giá các cơ chế Content Trust, seccomp, namespace và authorization plugin để tăng cường bảo mật khi build và triển khai container.

Các tiêu chí CIS được kiểm tra: 4.1–4.12, 2.10, 2.13, 2.15, 2.18, 2.19, 3.17–3.22.

– Container Runtime Security Audit

Module này tập trung kiểm tra bảo mật trong quá trình container runtime hoạt động, bao gồm network isolation, container privilege, resource limitation, filesystem isolation và healthcheck.

Mục tiêu là giảm thiểu rủi ro khai thác khi container đang chạy và đảm bảo container hoạt động trong môi trường bị giới hạn quyền truy cập và tài nguyên.

Các tiêu chí CIS được kiểm tra: 5.1–5.32.

2.2. Manual Remediation

Sau quá trình audit, hệ thống được hardening thủ công nhằm cải thiện mức độ bảo mật của hệ thống và giảm thiểu các rủi ro được phát hiện trong quá trình kiểm tra. Các thay đổi được áp dụng trực tiếp vào Dockerfile, Docker Compose và cấu hình runtime của container.

Các biện pháp remediation được triển khai bao gồm:

- Chạy container bằng non-root user thay vì root mặc định.
- Bật `no-new-privileges` để hạn chế privilege escalation.
- Thêm `HEALTHCHECK` nhằm giám sát trạng thái container.
- Giảm số lượng exposed ports và hạn chế publish port không cần thiết.
- Tách biệt internal network giữa các service.
- Loại bỏ các package và development tool không cần thiết trong image.
- Hạn chế mount các thư mục nhạy cảm từ host vào container.
- Sử dụng environment variable và GitHub Secrets để quản lý thông tin nhạy cảm.
- Áp dụng network isolation giữa Nginx, PHP-FPM, MySQL và Redis.
- Hardening container runtime bằng seccomp và security options.

3. Phương pháp thực hiện Automation

Sau khi hoàn thành quá trình manual hardening, hệ thống automation được xây dựng nhằm chuẩn hóa quy trình triển khai Docker Infrastructure, giảm thao tác thủ công và đảm bảo cấu hình bảo mật luôn được áp dụng đồng nhất giữa các lần deploy.

3.1. Mục tiêu

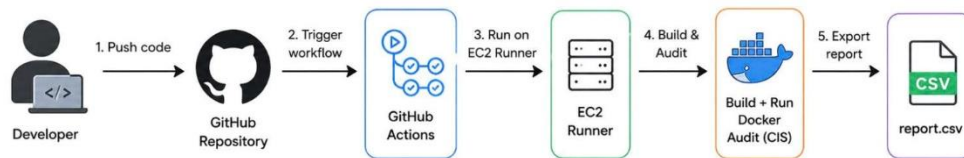
Hệ thống automation được thiết kế nhằm:

- Tự động build Docker image
- Tự động deploy hệ thống lên EC2
- Tự động cấu hình environment variables
- Tự động restart container khi có thay đổi source code
- Tự động thực hiện CIS Docker Benchmark Audit
- Giảm sai sót do thao tác thủ công
- Đảm bảo tính nhất quán giữa các lần triển khai
- Rút ngắn thời gian triển khai và kiểm tra hệ thống

Thành phần	Chức năng
GitHub Actions	CI/CD Pipeline dùng để tự động hóa workflow triển khai
GitHub Secrets	Lưu trữ thông tin nhạy cảm như mật khẩu database
Self-hosted Runner	Thực thi workflow trực tiếp trên EC2

Docker Compose	Quản lý và orchestration các container
Bash Audit Script	Tự động kiểm tra CIS Docker Benchmark

3.2. Luồng hoạt động Automation



Quy trình automation hoạt động theo mô hình CI/CD tự động. Khi source code được push lên GitHub Repository, GitHub Actions sẽ tự động trigger workflow triển khai. Self-hosted Runner chạy trên EC2 nhận workflow và tiến hành pull source code mới nhất từ repository.

Sau đó workflow tự động tạo file .env từ GitHub Secrets nhằm cung cấp các biến môi trường cần thiết cho hệ thống mà không cần hardcode thông tin nhạy cảm trong source code. Docker Compose tiếp tục thực hiện quá trình rebuild image và restart các container để cập nhật phiên bản mới của ứng dụng.

Khi hệ thống khởi động hoàn tất, các Bash Script kiểm tra CIS Docker Benchmark sẽ được thực thi tự động nhằm đánh giá mức độ hardening và cấu hình bảo mật của Docker daemon, image và container runtime. Kết quả audit được hiển thị trực tiếp trên workflow log của GitHub Actions, giúp dễ dàng theo dõi trạng thái triển khai và kiểm tra bảo mật của hệ thống.

Chương III. TRIỂN KHAI HỆ THỐNG

1. Triển khai hạ tầng

1.1. Cài đặt Docker và Docker Compos

Cài đặt Docker:

```
curl -fsSL https://get.docker.com | sh
```

Kiểm tra Docker:

```
docker -version
docker compose version
```

Khởi động Docker:

```
sudo systemctl enable docker
sudo systemctl start docker
```

1.2. Cấu hình Security Group

Để cho phép người dùng truy cập website từ Internet, nhóm cấu hình Security Group trên AWS EC2:

Inbound rules (4)							
Q Search							
Security group rule ID	IP version	Type	Protocol	Port range	Source	Description	
sgr-0b53779f95ed0ff8a	IPv4	Custom TCP	TCP	8081	0.0.0.0/0	-	
sgr-0f8e27cad1afc86c1	IPv4	HTTPS	TCP	443	0.0.0.0/0	-	
sgr-045807239764d96ce	IPv4	SSH	TCP	22	27.74.138.52/32	-	
sgr-0f6b2ca533b75ca5c	IPv4	HTTP	TCP	80	0.0.0.0/0	-	

Ngoài các port hệ thống, container Nginx được publish thông qua Docker Compose:

```
ports:
  - "8081:8080"
```

Cho phép truy cập ứng dụng thông qua:

```
http://<public-ip>:8081
```

1.3. Triển khai Docker Compose

Hệ thống được điều phối bằng Docker Compose với bốn service chính gồm Nginx, PHP-FPM Application, MySQL và Redis.

Nginx

- Đóng vai trò reverse proxy và public endpoint.
- Tiếp nhận request từ Internet và forward sang PHP-FPM.
- Chạy bằng non-root user nhằm giảm privilege escalation.
- Được cấu hình healthcheck để giám sát trạng thái hoạt động.

App (PHP-FPM)

- Thực thi ứng dụng web PHP.
- Kết nối tới MySQL và Redis thông qua internal Docker network.
- Sử dụng non-root user để tăng cường bảo mật container runtime.

MySQL

- Cung cấp dịch vụ lưu trữ dữ liệu.
- Sử dụng Docker volume để đảm bảo tính persistent của dữ liệu.
- Thông tin xác thực được quản lý thông qua environment variables thay vì hardcoded.

Redis

- Được sử dụng làm cache service.
- Chỉ hoạt động trong internal network và không expose trực tiếp ra Internet.

Triển khai hệ thống bằng Docker Compose:

```
docker compose up -d --build
```

Kiểm tra trạng thái container:

```
docker ps
```

Việc sử dụng Docker Compose giúp chuẩn hóa quá trình triển khai, giảm sai sót cấu hình thủ công và hỗ trợ automation trong CI/CD pipeline.

1.4. Quản lý biến môi trường

Thông tin nhạy cảm không được hardcoded trong source code mà được lưu thông qua file .env và GitHub Secrets.

Docker Compose sử dụng:

```
environment:  
  MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
```

Cách tiếp cận này giúp:

- Tăng tính bảo mật cho hệ thống.

- Tránh lộ credentials trên GitHub Repository.
- Dễ dàng thay đổi cấu hình giữa các môi trường triển khai.
- Hỗ trợ CI/CD automation hiệu quả hơn.

1.5. Thiết lập GitHub Runner

Để phục vụ CI/CD Automation, GitHub Self-hosted Runner được triển khai trực tiếp trên EC2 nhằm cho phép GitHub Actions thực hiện deploy ngay trên máy chủ runtime.

Khởi tạo thư mục runner:

```
mkdir actions-runner
cd actions-runner
```

Tải GitHub Runner:

```
curl -o actions-runner.tar.gz -L <runner-url>
```

Giải nén runner:

```
tar xzf actions-runner.tar.gz
```

Cấu hình và khởi động runner:

```
./config.sh
sudo ./svc.sh install
sudo ./svc.sh start
```

Self-hosted GitHub Runner được triển khai dưới dạng systemd service nhằm đảm bảo runner có thể tự động khởi động cùng hệ thống, duy trì hoạt động ổn định và hỗ trợ quá trình CI/CD Automation liên tục trên EC2.

```
ubuntu@ip-10-0-1-48:~$ systemctl status actions.runner.*
● actions.runner.dghnn-NT542.Q22-Group1-CIS-Docker.ip-10-0-1-48.service - GitHub Actions Runner (dghnn-NT542.Q22-Group1-CIS-Docker.ip-10-0-1-48.service)
   Loaded: loaded (/etc/systemd/system/actions.runner.dghnn-NT542.Q22-Group1-CIS-Docker.ip-10-0-1-48.service; vendor preset: enabled)
   Active: active (running) since Mon 2026-05-18 18:50:18 UTC; 18min ago
     Main PID: 519 (runsv.sh)
       Tasks: 20 (limit: 1013)
      Memory: 79.0M (peak: 127.8M swap: 12.2M swap peak: 12.2M)
         CPU: 3.801s
    CGroup: /system.slice/actions.runner.dghnn-NT542.Q22-Group1-CIS-Docker.ip-10-0-1-48.service
            └─519 /bin/bash /home/ubuntu/actions-runner/runsv.sh
               └─559 ./externals/node20/bin/node ./bin/RunnerService.js
                  └─825 /home/ubuntu/actions-runner/bin/Runner.Listener run --startuptype service

May 18 18:50:18 ip-10-0-1-48 systemd[1]: Started actions.runner.dghnn-NT542.Q22-Group1-CIS-Docker.ip-10-0-1-48.service.
May 18 18:50:19 ip-10-0-1-48 runsv.sh[519]: .path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
May 18 18:50:20 ip-10-0-1-48 runsv.sh[559]: Starting Runner listener with startup type: service
May 18 18:50:20 ip-10-0-1-48 runsv.sh[559]: Started listener process, pid: 825
May 18 18:50:20 ip-10-0-1-48 runsv.sh[559]: Started running service
May 18 18:50:25 ip-10-0-1-48 runsv.sh[559]: √ Connected to GitHub
May 18 18:50:25 ip-10-0-1-48 runsv.sh[559]: Current runner version: '2.334.0'
May 18 18:50:26 ip-10-0-1-48 runsv.sh[559]: 2026-05-18 18:50:25Z: Listening for Jobs
```

Sau khi cấu hình hoàn tất, GitHub Actions có thể tự động trigger workflow, pull source code mới nhất, build lại Docker image và triển khai trực tiếp lên EC2 mà không cần thao tác thủ công.

1.6. Kiểm tra hoạt động hệ thống

Sau khi triển khai hoàn tất, tiến hành kiểm tra trạng thái hoạt động của toàn bộ hệ thống container.

Kiểm tra container runtime:

```
docker ps
```

```
ubuntu@ip-10-0-1-48:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
4a31ade900c9   nt542q22-group1-cis-docker-nginx   "/docker-entrypoint.s..." 3 minutes ago  Up 2 minutes (healthy)   80/tcp, 0.0.0.0:8081->8080/t
-cis-docker-nginx-1
3d1e57eb1de7   redis:alpine                       "docker-entrypoint.s..." 3 minutes ago  Up 2 minutes (healthy)   6379/tcp
-cis-docker-redis-1
1f5e090a8442   nt542q22-group1-cis-docker-app     "docker-php-entrypoi..." 3 minutes ago  Up 2 minutes (healthy)   9000/tcp
-cis-docker-app-1
4ace078f73c0   mysql:8                             "docker-entrypoint.s..." 3 minutes ago  Up 2 minutes (healthy)   3306/tcp, 33060/tcp
-cis-docker-db-1
```

Kiểm tra phản hồi ứng dụng:

```
curl localhost:8081
```

```
ubuntu@ip-10-0-1-48:~$ curl localhost:8081
<!DOCTYPE html>
<html lang="en">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Secure Docker Infrastructure</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">

<link rel="stylesheet" href="assets/css/style.css">
</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark">

<div class="container">

<a class="navbar-brand" href="#">
CIS Docker Dashboard
</a>
```

Ngoài ra có thể truy cập trực tiếp từ trình duyệt thông qua:

```
http://<public-ip>:8081
```



Kết quả kiểm tra cho thấy các container hoạt động ổn định, Nginx forward request thành công tới PHP-FPM và ứng dụng có thể kết nối tới MySQL cùng Redis thông qua internal Docker network.

2. Triển khai Manual

2.1. Module 1:

1.1.1 Ensure a separate partition for containers is created - /var/lib/docker

Audit:

Mục tiêu: Xác nhận thư mục /var/lib/docker được mount trên một phân vùng riêng nhằm cô lập dữ liệu container, image, volume và filesystem của Docker khỏi hệ điều hành. Điều này giúp tránh tình trạng Docker sử dụng toàn bộ dung lượng ổ đĩa hệ thống và tăng tính ổn định cho host.

Tiến hành kiểm tra bằng lệnh:

```
grep '/var/lib/docker' /proc/mounts
mountpoint -q /var/lib/docker && echo "Mounted" || echo "Not Mounted"
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ grep '/var/lib/docker' /proc/mounts
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ mountpoint -q /var/lib/docker && echo "Mounted" || echo "Not Mounted"
Not Mounted
```

grep không trả về mount point riêng

- mountpoint hiển thị Not Mounted

Điều này cho thấy thư mục /var/lib/docker đang sử dụng chung phân vùng với hệ điều hành và chưa đáp ứng yêu cầu CIS Docker Benchmark – Control 1.1.1.

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Không có mount point riêng cho /var/lib/docker
- Docker data-root dùng chung với root filesystem /

→ Điều này có thể dẫn đến việc dữ liệu Docker làm đầy ổ đĩa hệ thống và ảnh hưởng đến tính sẵn sàng của host.

1.1.2 Restrict network traffic between containers on the default bridge

Audit:

Mục tiêu: Kiểm tra các user thuộc group docker nhằm đảm bảo chỉ những user đáng tin cậy mới có quyền điều khiển Docker daemon. Các thành viên trong group docker có quyền gần tương đương root do có thể thao tác trực tiếp với container và tài nguyên hệ thống.

Tiến hành kiểm tra bằng lệnh:

```
getent group docker
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ getent group docker
docker:x:999:van1768
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$
```

Kết quả trên cho thấy hiện tại các user ubuntu và van1768 đang có quyền sử dụng Docker.

Hệ thống được xem là **PASS** nếu:

- Chỉ có các user quản trị hợp lệ thuộc group docker
- Không tồn tại user lạ hoặc không cần thiết

Hệ thống sẽ bị đánh giá là **FAIL/WARN** nếu:

- Có quá nhiều user trong group docker
- Xuất hiện các user không đáng tin cậy như test, guest, hoặc user không phục vụ quản trị Docker

Điều này là do user thuộc group docker có thể:

- Chạy container với quyền cao
- Mount filesystem của host
- Tác động trực tiếp đến Docker daemon và hệ thống host

→ Vì vậy cần hạn chế số lượng user trong group docker để giảm nguy cơ privilege escalation và truy cập trái phép.

1.2.2 Ensure the version of Docker is up to date

Audit:

Mục tiêu: Xác nhận Docker Engine đang sử dụng phiên bản mới hoặc không chứa lỗ hổng nghiêm trọng đã biết. Việc sử dụng Docker version quá cũ có thể dẫn đến nguy cơ tồn tại vulnerability ảnh hưởng đến toàn bộ container runtime.

Tiến hành kiểm tra bằng lệnh:

```
docker version
```

Lệnh này hiển thị:

- Client Version
- Server (Docker Engine) Version

Hệ thống được xem là **PASS** nếu:

- Docker đang sử dụng version mới hoặc gần mới
- Không tồn tại vulnerability nghiêm trọng đã biết

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Docker version quá cũ
- Đã deprecated hoặc chứa lỗ hổng bảo mật nghiêm trọng

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ docker version
Client: Docker Engine - Community
Version: 29.4.0
API version: 1.54
Go version: go1.26.1
Git commit: 9d7ad9f
Built: Tue Apr 7 08:35:18 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 29.4.0
API version: 1.54 (minimum version 1.40)
Go version: go1.26.1
Git commit: daa0cb7
Built: Tue Apr 7 08:35:18 2026
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: v2.2.3
GitCommit: 77c84241c7cbdd9b4eca2591793e3d4f4317c590
runc:
Version: 1.3.5
GitCommit: v1.3.5-0-g488fc13e
docker-init:
Version: 0.19.0
GitCommit: de40ad0
```

2.6 Ensure aufs storage driver is not used

Audit:

Mục tiêu: Xác nhận Docker không sử dụng storage driver aufs. Driver này hiện không còn được khuyến nghị sử dụng do hạn chế về hiệu năng và khả năng bảo trì.

Tiến hành kiểm tra bằng lệnh:

```
docker info --format '{{ .Driver }}'
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ docker info --format '{{ .Driver }}'
overlayfs
```

Hệ thống được xem là **PASS** nếu:

- Storage Driver khác aufs

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Storage Driver là aufs

→ Hệ thống hiện sử dụng overlayfs nên đáp ứng CIS Docker Benchmark – Control 2.6.

2.7 Ensure devicemapper storage driver is not used

Audit:

Mục tiêu: Xác nhận Docker không sử dụng devicemapper storage driver. Driver này đã deprecated và bị loại bỏ khỏi Docker Engine v25 do không còn đảm bảo tính ổn định và bảo mật.

Tiến hành kiểm tra bằng lệnh:

```
docker info --format '{{ .Driver }}'
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ docker info --format '{{ .Driver }}'
```

overlayfs

Hệ thống được xem là **PASS** nếu:

- Storage Driver khác devicemapper

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Storage Driver là devicemapper

→ Hệ thống hiện sử dụng overlayfs nên đáp ứng CIS Docker Benchmark – Control 2.7.

2.11 Ensure that authorization for Docker client commands is centralized

Audit:

Mục tiêu: Xác nhận hệ thống không cấu hình cgroup-parent không cần thiết. Việc cấu hình sai có thể khiến container chia sẻ tài nguyên không kiểm soát và ảnh hưởng hiệu năng hệ thống.

Tiến hành kiểm tra bằng lệnh:

```
cat /etc/docker/daemon.json
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ grep "cgroup-parent" /etc/docker/daemon.json
grep: /etc/docker/daemon.json: No such file or directory
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ ps -ef | grep dockerd
root      1919      1  0 09:05 ?        00:00:01 /usr/bin/dockerd -H fd:// --containerd=/run/
containerd/containerd.sock
van1768   3061   2436  0 09:31 pts/0    00:00:00 grep --color=auto dockerd
```

- Không tồn tại file /etc/docker/daemon.json
- Không có cấu hình cgroup-parent

Hệ thống được xem là **PASS** nếu:

- Không cấu hình cgroup-parent
- Hoặc cấu hình hợp lý theo policy quản trị

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Cấu hình cgroup-parent sai hoặc không cần thiết

→ Hệ thống hiện dùng cấu hình mặc định của Docker nên đáp ứng CIS Docker Benchmark – Control 2.11.

2.12 Ensure the default ulimit is configured appropriately

Audit:

Mục tiêu: Xác nhận hệ thống không thay đổi dm.basesize. Việc tăng base device size có thể khiến container chiếm quá nhiều dung lượng lưu trữ và gây ảnh hưởng đến hệ thống.

Tiến hành kiểm tra bằng lệnh:

```
cat /etc/docker/daemon.json
docker info
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ grep "storage-opt dm.basesize" /etc/docker/daemon.json
grep: /etc/docker/daemon.json: No such file or directory
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ docker info | grep "Storage Driver"
Storage Driver: overlayfs
```

- Không tồn tại cấu hình dm.basesize
- Storage Driver hiện tại là overlayfs

Hệ thống được xem là **PASS** nếu:

- Không cấu hình dm.basesize

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Có cấu hình dm.basesize không cần thiết

→ Hệ thống hiện không sử dụng devicemapper và không cấu hình dm.basesize, đáp ứng CIS Docker Benchmark – Control 2.12.

3.1 Ensure that docker.service file ownership is set to root:root

Audit:

Mục tiêu: Xác nhận file docker.service thuộc sở hữu root:root nhằm ngăn user thường chỉnh sửa cấu hình Docker daemon.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %U:%G /lib/systemd/system/docker.service
```

Kết quả:

Hệ thống được xem là **PASS** nếu:

- Owner là root:root

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Owner khác root:root

→ File docker.service hiện thuộc sở hữu đúng chuẩn CIS.

3.2 Ensure that docker.service file permissions are set to 644 or more restrictive

Audit:

Mục tiêu: Xác nhận permission của file docker.service là 644 hoặc chặt hơn nhằm ngăn user thường sửa đổi cấu hình Docker daemon.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %a /lib/systemd/system/docker.service
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ systemctl show -p FragmentPath docker.service
FragmentPath=/lib/systemd/system/docker.service
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%a" /lib/systemd/system/docker.service
644
```

Hệ thống được xem là **PASS** nếu:

- Permission ≤ 644

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Permission lớn hơn 644

→ Chỉ root có quyền ghi file docker.service, đáp ứng chuẩn CIS.

3.3 Ensure that docker.socket file ownership is set to root:root

Audit:

Mục tiêu: Xác nhận file docker.socket thuộc sở hữu root:root nhằm hạn chế truy cập trái phép vào Docker daemon.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %U:%G /lib/systemd/system/docker.socket
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ systemctl show -p FragmentPath docker.socket
FragmentPath=/lib/systemd/system/docker.socket
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%U %G" /lib/systemd/system/docker.socket
root root
```

Hệ thống được xem là **PASS** nếu:

- Owner là root:root

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Owner khác root:root

→ File docker.socket đáp ứng yêu cầu CIS Docker Benchmark – Control 3.3.

3.5 Ensure that /etc/docker directory ownership is set to root:root

Audit:

Mục tiêu: Xác nhận thư mục /etc/docker thuộc sở hữu root:root nhằm bảo vệ các file cấu hình, certificate và key quan trọng của Docker.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %U:%G /etc/docker
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ sudo chown root:root /etc/docker
[sudo] password for van1768:
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%U:%G" /etc/docker
root:root
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%U:%G" /etc/docker | grep -v root:root
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$
```

Hệ thống được xem là **PASS** nếu:

- Owner là root:root

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Owner khác root:root

→ Thư mục /etc/docker đáp ứng chuẩn CIS Docker Benchmark – Control 3.5.

3.6 Ensure that /etc/docker directory permissions are set to 755 or more restrictive

Audit:

Mục tiêu: Xác nhận permission của thư mục /etc/docker là 755 hoặc chặt hơn nhằm ngăn user thường chỉnh sửa cấu hình Docker.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %a /etc/docker
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%a" /etc/docker
755
```

Hệ thống được xem là **PASS** nếu:

- Permission ≤ 755

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Permission lớn hơn 755

→ Hệ thống đáp ứng CIS Docker Benchmark – Control 3.6.

3.15 Ensure that Docker socket file ownership is set to root:docker

Audit:

Mục tiêu: Xác nhận file /var/run/docker.sock thuộc sở hữu root:docker nhằm đảm bảo chỉ root và group docker mới có quyền truy cập Docker daemon.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %U:%G /var/run/docker.sock
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%U:%G" /var/run/docker.sock | grep -v root:docker
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$
```

Hệ thống được xem là **PASS** nếu:

- Owner là root
- Group là docker

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Ownership khác root:docker

→ File docker.sock đáp ứng CIS Docker Benchmark – Control 3.15.

3.16 Ensure that Docker socket file permissions are set to 660 or more restrictive

Audit:

Mục tiêu: Xác nhận permission của file /var/run/docker.sock là 660 hoặc chặt hơn nhằm hạn chế truy cập trái phép vào Docker daemon.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %a /var/run/docker.sock
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%a" /var/run/docker.sock
660
```

Hệ thống được xem là **PASS** nếu:

- Permission ≤ 660

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Permission lớn hơn 660

→ Chỉ root và group docker có quyền truy cập Docker daemon.

3.23 Ensure that containerd socket file ownership is set to root:root

Audit:

Mục tiêu: Xác nhận file /run/containerd/containerd.sock thuộc sở hữu root:root nhằm bảo vệ container runtime khỏi truy cập trái phép.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %U:%G /run/containerd/containerd.sock
```

Kết quả:

```
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$ stat -c "%U:%G" /run/containerd/containerd.sock | grep -v root:root
van1768@van1768:~/NT542.Q22-Group1-CIS-Docker$
```

Hệ thống được xem là **PASS** nếu:

- Owner là root:root

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Owner khác root:root

→ File containerd.sock đáp ứng chuẩn CIS Docker Benchmark – Control 3.23.

3.24 *Ensure that containerd socket file permissions are set to 660 or more restrictive*

Audit:

Mục tiêu: Xác nhận permission của file `/run/containerd/containerd.sock` là 660 hoặc chặt hơn nhằm hạn chế user ngoài thao túng container runtime.

Tiến hành kiểm tra bằng lệnh:

```
stat -c %a /run/containerd/containerd.sock
```

Kết quả:

```
van1768@van1768:~/NT542-Q22-Group1-CIS-Docker$ stat -c "%a" /run/containerd/containerd.sock
660
```

Hệ thống được xem là **PASS** nếu:

- Permission ≤ 660

Hệ thống sẽ bị đánh giá là **FAIL** nếu:

- Permission lớn hơn 660

→ File `containerd.sock` đáp ứng CIS Docker Benchmark – Control 3.24.

2.2. Module 2:

1.1.3 *Ensure auditing is configured for the Docker daemon*

Audit:

Mục tiêu: Xác nhận Docker daemon đang được theo dõi bởi audit framework của Linux. Vì dockerd hoạt động với quyền root và chịu trách nhiệm quản lý toàn bộ container, image, network và volume nên việc audit giúp theo dõi hành vi quản trị, phát hiện truy cập trái phép và hỗ trợ điều tra sự cố bảo mật.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /usr/bin/dockerd
-w /usr/bin/dockerd -p wa -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả trả về được xem là đạt chuẩn và an toàn. Điều này cho thấy Docker daemon đang được giám sát bởi auditd và các hành vi liên quan đến file thực thi dockerd sẽ được ghi log phục vụ monitoring và forensic.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh không trả về kết quả nào. Điều đó đồng nghĩa với việc Docker daemon chưa được cấu hình audit và mọi hoạt động của daemon sẽ không được ghi nhận trong audit log, làm giảm khả năng phát hiện tấn công hoặc truy vết sự cố bảo mật.

1.1.4 *Ensure auditing is configured for Docker files and directories- /run/containerd*

Audit:

Mục tiêu: Xác nhận thư mục `/run/containerd` đang được audit để theo dõi các thay đổi

liên quan đến container runtime. Thư mục này lưu trữ thông tin trạng thái và hoạt động của container, vì vậy nếu bị chỉnh sửa trái phép có thể ảnh hưởng trực tiếp đến quá trình vận hành và bảo mật của Docker.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /run/containerd
-w /run/containerd/containerd.sock -p rwx -k docker
-w /run/containerd -p rwa -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả trả về được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang theo dõi các hành động đọc, ghi hoặc thay đổi thuộc tính liên quan đến thư mục /run/containerd, giúp phát hiện truy cập bất thường và hỗ trợ forensic khi có sự cố.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh không trả về kết quả nào. Điều đó đồng nghĩa với việc thư mục runtime của container chưa được audit và các thay đổi quan trọng liên quan đến container có thể diễn ra mà không được ghi nhận trong audit log.

1.1.5 Ensure auditing is configured for Docker files and directories- /var/lib/docker

Audit:

Mục tiêu: Xác nhận thư mục /var/lib/docker hoặc thư mục data-root tùy chỉnh đang được audit để theo dõi các thay đổi liên quan đến dữ liệu container, image, volume và filesystem của Docker. Đây là thư mục quan trọng vì chứa toàn bộ dữ liệu runtime và storage của Docker, do đó cần được giám sát để phát hiện truy cập hoặc thay đổi trái phép.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /var/lib/docker
-w /var/lib/docker -p rwa -k docker
-a never,exit -S all -F dir=/var/lib/docker/volumes
-a never,exit -S all -F dir=/var/lib/docker/overlay2
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát các hành động đọc, ghi và thay đổi thuộc tính đối với thư mục dữ liệu Docker.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh kiểm tra không trả về audit rule nào cho /var/lib/docker hoặc data-root. Điều đó đồng nghĩa với việc dữ liệu Docker không được audit và các thay đổi liên quan đến container hoặc image có thể diễn ra mà không được ghi nhận trong audit log.

1.1.6 Ensure auditing is configured for Docker files and directories- /etc/docker

Audit:

Mục tiêu: Xác nhận thư mục /etc/docker đang được audit để theo dõi các thay đổi liên quan đến cấu hình Docker daemon, chứng chỉ TLS, khóa bảo mật và các thiết lập quan trọng của Docker. Vì Docker daemon hoạt động với quyền root, mọi thay đổi trái

phép trong thư mục này có thể ảnh hưởng trực tiếp đến bảo mật và hoạt động của hệ thống container.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /etc/docker
-w /etc/docker -p rwx -k docker
-w /etc/docker/daemon.json -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả trả về được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang theo dõi các thay đổi liên quan đến thư mục cấu hình Docker và mọi hành động truy cập hoặc chỉnh sửa sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh không trả về kết quả nào. Điều đó đồng nghĩa với việc thư mục /etc/docker chưa được audit và các thay đổi đối với cấu hình, chứng chỉ hoặc khóa bảo mật Docker có thể diễn ra mà không được ghi log.

1.1.7 Ensure auditing is configured for Docker files and directories- docker.service

Audit:

Mục tiêu: Xác nhận file docker.service đang được audit để theo dõi các thay đổi liên quan đến cấu hình khởi động Docker daemon. File này có thể chứa các tham số vận hành quan trọng của Docker như socket, cgroup driver, logging, TLS hoặc các tùy chọn bảo mật khác. Vì Docker daemon chạy với quyền root, mọi thay đổi trái phép trong file service có thể ảnh hưởng trực tiếp đến toàn bộ môi trường container.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep docker.service
-w /lib/systemd/system/docker.service -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang theo dõi các thay đổi liên quan đến file service của Docker daemon và mọi hành động chỉnh sửa sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu file docker.service tồn tại nhưng lệnh kiểm tra không trả về audit rule tương ứng. Điều đó đồng nghĩa với việc các thay đổi đối với cấu hình service của Docker có thể diễn ra mà không được ghi log.

1.1.8 Ensure auditing is configured for Docker files and directories- containerd.sock

Audit:

Mục tiêu: Xác nhận file containerd.sock đang được audit để theo dõi các hoạt động liên quan đến container runtime. Socket này được Docker daemon sử dụng để giao tiếp với containerd, thành phần chịu trách nhiệm quản lý lifecycle của container. Vì Docker và containerd hoạt động với quyền cao trên hệ thống, việc giám sát socket này giúp phát hiện truy cập trái phép hoặc thay đổi bất thường.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep containerd.sock
-w /run/containerd/containerd.sock -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát các hoạt động liên quan đến socket giao tiếp của containerd và mọi truy cập hoặc thay đổi sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu file containerd.sock tồn tại nhưng lệnh kiểm tra không trả về audit rule tương ứng. Điều đó đồng nghĩa với việc các hoạt động liên quan đến container runtime có thể diễn ra mà không được ghi log.

1.1.9 Ensure auditing is configured for Docker files and directories- docker.sock

Audit:

Mục tiêu: Xác nhận file docker.sock đang được audit để theo dõi các hoạt động giao tiếp với Docker daemon. Socket này cho phép Docker client và các tiến trình khác giao tiếp trực tiếp với Docker daemon. Vì Docker daemon hoạt động với quyền root, bất kỳ ai có quyền truy cập vào docker.sock đều có thể kiểm soát gần như toàn bộ hệ thống container, do đó socket này cần được giám sát chặt chẽ.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep docker.sock
-w /var/run/docker.sock -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát các hoạt động liên quan đến Docker socket và mọi truy cập hoặc thay đổi sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu file docker.sock tồn tại nhưng lệnh kiểm tra không trả về audit rule tương ứng. Điều đó đồng nghĩa với việc các hoạt động truy cập Docker daemon thông qua socket có thể diễn ra mà không được ghi log, làm tăng nguy cơ lạm dụng quyền root hoặc kiểm soát container trái phép.

1.1.10 Ensure auditing is configured for Docker files and directories/etc/default/docker

Audit:

Mục tiêu: Xác nhận file /etc/default/docker đang được audit để theo dõi các thay đổi liên quan đến tham số khởi động và cấu hình mặc định của Docker daemon. File này có thể chứa các tùy chọn quan trọng ảnh hưởng đến hành vi của Docker như network, storage driver, logging hoặc các tham số bảo mật. Vì Docker daemon hoạt động với quyền root, mọi thay đổi trái phép đối với file này có thể ảnh hưởng trực tiếp đến bảo mật của hệ thống container.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /etc/default/docker
-w /etc/default/docker -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát các thay đổi liên quan đến file cấu hình mặc định của Docker và mọi hành động chỉnh sửa sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh kiểm tra không trả về kết quả nào. Điều đó đồng nghĩa với việc file /etc/default/docker chưa được audit và các thay đổi cấu hình quan trọng của Docker daemon có thể diễn ra mà không được ghi log.

1.1.11 Ensure auditing is configured for Docker files and directories /etc/docker/daemon.json

Audit:

Mục tiêu: Xác nhận file /etc/docker/daemon.json đang được audit để theo dõi các thay đổi liên quan đến cấu hình Docker daemon. File này chứa nhiều thiết lập quan trọng như logging, TLS, storage driver, registry, namespace, cgroup và các tùy chọn bảo mật khác. Vì Docker daemon hoạt động với quyền root, việc thay đổi trái phép file cấu hình này có thể ảnh hưởng trực tiếp đến toàn bộ môi trường container và bảo mật hệ thống.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /etc/docker/daemon.json
-w /etc/docker/daemon.json -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát các thay đổi liên quan đến file cấu hình chính của Docker daemon và mọi hành động chỉnh sửa sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh kiểm tra không trả về kết quả nào. Điều đó đồng nghĩa với việc file /etc/docker/daemon.json chưa được audit và các thay đổi cấu hình quan trọng của Docker có thể diễn ra mà không được ghi log.

1.1.12 Ensure auditing is configured for Docker files and directories - /etc/containerd/config.toml

Audit:

Mục tiêu: Xác nhận file /etc/containerd/config.toml đang được audit để theo dõi các thay đổi liên quan đến cấu hình của containerd. File này chứa nhiều tham số quan trọng như runtime, snapshotter, namespace, socket và các thiết lập vận hành của container runtime. Vì containerd hoạt động với quyền cao trên hệ thống và đóng vai trò quản lý container runtime cho Docker, việc giám sát file cấu hình này giúp phát hiện thay đổi trái phép và hỗ trợ điều tra sự cố bảo mật.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /etc/containerd/config.toml
-w /etc/containerd/config.toml -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát các thay đổi liên quan đến file cấu hình của containerd và mọi hành động chỉnh sửa sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu file /etc/containerd/config.toml tồn tại nhưng lệnh kiểm tra không trả về kết quả nào. Điều đó đồng nghĩa với việc các thay đổi cấu hình liên quan đến container runtime có thể diễn ra mà không được ghi log.

1.1.13 Ensure auditing is configured for Docker files and directories - /etc/sysconfig/docker

Audit:

Mục tiêu: Xác nhận file /etc/sysconfig/docker đang được audit để theo dõi các thay đổi liên quan đến cấu hình Docker daemon trên các hệ điều hành dựa trên CentOS hoặc RHEL. File này có thể chứa các tham số vận hành quan trọng của Docker như network, storage, logging hoặc các tùy chọn bảo mật khác. Vì Docker daemon hoạt động với quyền root, việc thay đổi trái phép file này có thể ảnh hưởng trực tiếp đến bảo mật và hoạt động của hệ thống container.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /etc/sysconfig/docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát các thay đổi liên quan đến file cấu hình Docker và mọi hành động chỉnh sửa sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu file tồn tại nhưng lệnh kiểm tra không trả về audit rule tương ứng. Điều đó đồng nghĩa với việc các thay đổi cấu hình Docker có thể diễn ra mà không được ghi log.

Ghi chú: với hệ điều hành Ubuntu/Debian sẽ không có file này

1.1.14 Ensure auditing is configured for Docker files and directories - /usr/bin/containerd

Audit:

Mục tiêu: Xác nhận file /usr/bin/containerd đang được audit để theo dõi các thay đổi liên quan đến container runtime của Docker. containerd là thành phần chịu trách nhiệm quản lý lifecycle của container như tạo, chạy và dừng container. Vì tiến trình này hoạt động với quyền cao trên hệ thống, việc giám sát file thực thi giúp phát hiện thay đổi trái phép hoặc nguy cơ bị thay thế binary độc hại.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /usr/bin/containerd
-w /usr/bin/containerd -p rwx -k docker
-w /usr/bin/containerd-shim-runc-v2 -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát file thực thi containerd và mọi hành động truy cập hoặc chỉnh sửa liên quan sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh kiểm tra không trả về kết quả nào. Điều đó đồng nghĩa với việc file thực thi containerd chưa được audit và các thay đổi quan trọng liên quan đến container runtime có thể diễn ra mà không được ghi log.

1.1.15 Ensure auditing is configured for Docker files and directories - /usr/bin/containerd-shim

Audit:

Mục tiêu: Xác nhận file /usr/bin/containerd-shim đang được audit để theo dõi các thay đổi liên quan đến tiến trình trung gian giữa containerd và container runtime. containerd-shim có vai trò duy trì container hoạt động độc lập với tiến trình containerd, đồng thời quản lý stdin/stdout và trạng thái container. Vì tiến trình này liên quan trực tiếp đến runtime của container và hoạt động với quyền cao trên hệ thống, việc giám sát file thực thi giúp phát hiện hành vi thay đổi trái phép hoặc nguy cơ bị thay thế binary độc hại.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /usr/bin/containerd-shim
-w /usr/bin/containerd-shim-runc-v2 -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát file thực thi containerd-shim và mọi hành động truy cập hoặc chỉnh sửa liên quan sẽ được ghi nhận bởi auditd.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh kiểm tra không trả về kết quả nào. Điều đó đồng nghĩa với việc file thực thi containerd-shim chưa được audit và các thay đổi liên quan đến container runtime có thể diễn ra mà không được ghi log.

1.1.16 Ensure auditing is configured for Docker files and directories - /usr/bin/containerd-shim-runc-v1

Audit:

Mục tiêu: Xác nhận file /usr/bin/containerd-shim-runc-v1 đang được audit để theo dõi các thay đổi liên quan đến runtime shim sử dụng runc. Thành phần này đóng vai trò trung gian giữa containerd và OCI runtime (runc) trong quá trình quản lý container. Vì file này liên quan trực tiếp đến việc khởi chạy và duy trì container với quyền cao trên hệ thống, việc giám sát giúp phát hiện thay đổi trái phép hoặc nguy cơ thay thế binary độc hại.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /usr/bin/containerd-shim-runc-v1
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát file thực thi containerd-shim-runc-v1 và mọi hành động truy cập hoặc chỉnh sửa liên quan sẽ được ghi nhận bởi auditd.

Nếu file /usr/bin/containerd-shim-runc-v1 không tồn tại thì khuyến nghị này không áp dụng cho hệ thống hiện tại.

Hệ thống sẽ bị đánh giá là FAIL nếu file tồn tại nhưng lệnh kiểm tra không trả về audit rule tương ứng. Điều đó đồng nghĩa với việc các thay đổi liên quan đến runtime shim của container có thể diễn ra mà không được ghi log.

Ghi chú: Ubuntu phiên bản hiện tại chỉ dùng v2

X

1.1.17 Ensure auditing is configured for Docker files and directories - /usr/bin/containerd-shim-runc-v2

Audit:

Mục tiêu: Xác nhận file /usr/bin/containerd-shim-runc-v2 đang được audit để theo dõi các thay đổi liên quan đến runtime shim sử dụng runc phiên bản v2. Thành phần này được containerd sử dụng để quản lý lifecycle của container và giao tiếp với OCI runtime. Vì file này hoạt động với quyền cao trên hệ thống và ảnh hưởng trực tiếp đến quá trình thực thi container, việc giám sát giúp phát hiện thay đổi trái phép hoặc nguy cơ thay thế binary độc hại.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /usr/bin/containerd-shim-runc-v2
-w /usr/bin/containerd-shim-runc-v2 -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát file thực thi containerd-shim-runc-v2 và mọi hành động truy cập hoặc chỉnh sửa liên quan sẽ được ghi nhận bởi auditd.

Nếu file /usr/bin/containerd-shim-runc-v2 không tồn tại thì khuyến nghị này không áp dụng cho hệ thống hiện tại.

Hệ thống sẽ bị đánh giá là FAIL nếu file tồn tại nhưng lệnh kiểm tra không trả về audit rule tương ứng. Điều đó đồng nghĩa với việc các thay đổi liên quan đến runtime shim của container có thể diễn ra mà không được ghi log.

1.1.18 Ensure auditing is configured for Docker files and directories - /usr/bin/runc

Audit:

Mục tiêu: Xác nhận file /usr/bin/runc đang được audit để theo dõi các thay đổi liên quan đến OCI runtime của Docker. runc là thành phần trực tiếp tạo và chạy container dựa trên OCI specification. Vì runtime này hoạt động với quyền cao trên hệ thống và có quyền truy cập sâu đến namespace, cgroup và filesystem của container, việc giám sát file thực thi giúp phát hiện thay đổi trái phép hoặc nguy cơ bị thay thế bằng binary độc hại.

```
anvu@anvu-VirtualBox:~$ sudo auditctl -l | grep /usr/bin/runc
-w /usr/bin/runc -p rwx -k docker
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy hệ thống đang giám sát file thực thi runc và mọi hành động truy cập hoặc chỉnh sửa liên quan sẽ được ghi nhận bởi auditd.

Nếu file /usr/bin/runc không tồn tại thì khuyến nghị này không áp dụng cho hệ thống hiện tại.

Hệ thống sẽ bị đánh giá là FAIL nếu file tồn tại nhưng lệnh kiểm tra không trả về audit rule tương ứng. Điều đó đồng nghĩa với việc các thay đổi liên quan đến OCI runtime có thể diễn ra mà không được ghi log, làm giảm khả năng phát hiện tấn công hoặc thay đổi trái phép trong môi trường container.

2.1 Run the Docker daemon as a non-root user, if possible

Audit:

Mục tiêu: Xác nhận Docker daemon đang được vận hành bằng tài khoản non-root nhằm giảm thiểu rủi ro bảo mật khi daemon hoặc container runtime bị khai thác. Ở chế độ rootless, cả dockerd và container sẽ hoạt động bên trong user namespace mà không cần đặc quyền root, giúp hạn chế khả năng privilege escalation và giảm tác động khi xảy ra compromise.

```
anvu@anvu-VirtualBox:~$ sudo ps -fe | grep 'dockerd'
root      966      1  2 03:15 ?        00:00:31 /usr/bin/dockerd -H fd:// --
containerd=/run/containerd/containerd.sock
anvu     16028   15114  0 03:41 pts/2    00:00:00 grep --color=auto dockerd
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn. Điều này cho thấy Docker daemon đang được quản lý bởi một tài khoản non-root và hệ thống đang sử dụng chế độ rootless.

Nếu kết quả trả về:

```
root      1023      1  0 10:00 ?        00:00:05 dockerd
38
```

thì hệ thống chưa chạy Docker daemon ở chế độ rootless. Đây là cấu hình mặc định của Docker và bị đánh giá là chưa đáp ứng khuyến nghị bảo mật CIS vì daemon vẫn hoạt động với quyền root.

2.3 Ensure the logging level is set to 'info'

Audit:

Mục tiêu: Xác nhận Docker daemon đang được cấu hình với mức log info. Mức log này cho phép ghi nhận đầy đủ các sự kiện vận hành quan trọng của Docker nhưng không bao gồm debug log chi tiết. Việc tránh sử dụng mức debug trong môi trường production giúp giảm rò rỉ thông tin nhạy cảm và hạn chế việc tạo log quá lớn gây ảnh hưởng hiệu năng hoặc dung lượng lưu trữ.

```
anvu@anvu-VirtualBox:~$ sudo grep "log-level" /etc/docker/daemon.json
"log-level": "info",
anvu@anvu-VirtualBox:~$
```

Kết quả trả về:

"log-level": "info"

được xem là đạt chuẩn và an toàn. Điều này cho thấy Docker daemon đang sử dụng mức log phù hợp theo khuyến nghị của CIS Benchmark.

Nếu không có tham số log-level trong file cấu hình, hệ thống vẫn được xem là PASS vì Docker mặc định sử dụng mức log info.

Hệ thống sẽ bị đánh giá là FAIL nếu kết quả trả về các giá trị như: "log-level": "debug" hoặc Docker daemon được khởi chạy với: `dockerd --log-level="debug"`

Điều này đồng nghĩa với việc Docker đang ghi quá nhiều thông tin debug, có thể làm tăng nguy cơ lộ thông tin nhạy cảm và tạo lượng log lớn không cần thiết.

2.8 Ensure TLS authentication for Docker daemon is configured

Audit:

Mục tiêu: Xác nhận Docker daemon được cấu hình TLS authentication khi expose qua TCP port hoặc network socket. Vì Docker daemon hoạt động với quyền root, bất kỳ ai truy cập được vào Docker socket từ xa đều có thể kiểm soát container và hệ thống host. Việc sử dụng TLS giúp xác thực client, mã hóa kết nối và giới hạn chỉ các client sở hữu certificate hợp lệ mới có thể truy cập Docker daemon.

```
anvu@anvu-VirtualBox:~$ sudo grep "--tls" /etc/docker/daemon.json
grep: unrecognized option '--tls'
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
anvu@anvu-VirtualBox:~$
```

Kết quả trả về như sau được xem là đạt chuẩn và an toàn:

```
"tlsverify": true,
"tlscacert": "/etc/docker/certs/ca.pem",
"tlscert": "/etc/docker/certs/server-cert.pem",
"tlskey": "/etc/docker/certs/server-key.pem"
```

Hệ thống sẽ bị đánh giá là FAIL nếu:

Docker daemon được expose qua TCP port nhưng không bật TLS

thiếu các tham số: tlsverify, tlscacert, tlscert, tlskey, hoặc daemon được khởi chạy với cấu hình không mã hóa

Ghi chú: FAIL

X

2.14 Ensure centralized and remote logging is configured

Audit:

Mục tiêu: Xác nhận Docker daemon được cấu hình sử dụng centralized hoặc remote logging nhằm đảm bảo log container được lưu trữ an toàn và có thể giám sát tập trung. Việc gửi log đến syslog server, SIEM hoặc log collector giúp giảm nguy cơ mất log khi host gặp sự cố và hỗ trợ monitoring, incident response cũng như forensic hiệu quả hơn.

```
anvu@anvu-VirtualBox:~$ sudo docker info --format '{{.LoggingDriver}}'
syslog
anvu@anvu-VirtualBox:~$
```

Kết quả được xem là đạt chuẩn và an toàn nếu phù hợp với chính sách logging của hệ thống và hỗ trợ centralized/remote log management.

Hệ thống sẽ bị đánh giá là FAIL nếu kết quả trả về json-file và không có cơ chế centralized logging bổ sung. Đây là logging driver mặc định của Docker, chỉ lưu log cục bộ trên host dưới dạng JSON file và không đáp ứng yêu cầu quản lý log tập trung theo CIS Benchmark.

3.7 Ensure that registry certificate file ownership is set to root:root

Audit:

Mục tiêu: Xác nhận các file certificate trong thư mục /etc/docker/certs.d/<registry-name>/ được sở hữu bởi user root và group root. Các certificate này được sử dụng để xác thực kết nối TLS giữa Docker daemon và registry. Nếu người dùng ít đặc quyền hơn có thể chỉnh sửa các file này, attacker có thể thay thế certificate hoặc thực hiện tấn công giả mạo registry.

```
anvu@anvu-VirtualBox:~$ sudo stat -c %U:%G /etc/docker/certs.d/* | grep -v root:root
anvu@anvu-VirtualBox:~$
```

Kết quả không trả về giá trị nào được xem là đạt chuẩn và an toàn. Điều này cho thấy toàn bộ certificate file đều được sở hữu bởi root:root và người dùng thông thường không thể chỉnh sửa nội dung certificate.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh trả về các kết quả như user1:user1 hoặc docker:docker

Điều này đồng nghĩa với việc một hoặc nhiều certificate file không thuộc sở hữu của root, làm tăng nguy cơ bị thay đổi trái phép hoặc giả mạo kết nối TLS với Docker registry.

3.8 Ensure that registry certificate file permissions are set to 444 or more restrictively

Audit:

Mục tiêu: Xác nhận các file certificate trong thư mục /etc/docker/certs.d/<registry-name>/ được cấu hình permission an toàn ở mức 444 hoặc nghiêm ngặt hơn. Các certificate này được sử dụng để xác thực TLS với Docker registry, vì vậy việc giới hạn quyền truy cập giúp ngăn người dùng không đặc quyền sửa đổi hoặc ghi đè certificate.

```
anvu@anvu-VirtualBox:~$ sudo find /etc/docker/certs.d/ -type f -exec stat -c "%a %n" {} \;
444 /etc/docker/certs.d/test.local/ca.crt
anvu@anvu-VirtualBox:~$
```

Kết quả cho thấy certificate file chỉ có quyền đọc và không cho phép người dùng thông thường chỉnh sửa nội dung file.

Nếu thư mục /etc/docker/certs.d/ không tồn tại thì khuyến nghị này không áp dụng cho hệ thống hiện tại.

Hệ thống sẽ bị đánh giá là FAIL nếu kết quả trả về các permission quá rộng như: 644 /etc/docker/certs.d/registry.example.com/ca.crt hoặc: 666 /etc/docker/certs.d/registry.example.com/ca.crt

Điều này đồng nghĩa với việc certificate file có thể bị chỉnh sửa trái phép, làm tăng nguy cơ giả mạo registry hoặc tấn công TLS.

3.9 Ensure that TLS CA certificate file ownership is set to root:root

Audit:

Mục tiêu: Xác nhận file TLS CA certificate được cấu hình owner và group là root:root. File này được Docker daemon sử dụng để xác thực certificate của Docker server hoặc Docker registry thông qua CA certificate. Nếu file CA bị chỉnh sửa trái phép, attacker có thể thực hiện giả mạo TLS hoặc can thiệp vào quá trình xác thực kết nối bảo mật.

```
anvu@anvu-VirtualBox:~$ sudo stat -c %U:%G /etc/docker/certs/ca.pem | grep -v root:root
anvu@anvu-VirtualBox:~$
```

Kết quả không trả về giá trị nào được xem là đạt chuẩn và an toàn. Điều này cho thấy file TLS CA certificate đang được sở hữu bởi root:root và người dùng không đặc quyền không thể thay đổi nội dung file.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh trả về các kết quả như docker:docker hoặc user1:user1

Điều này đồng nghĩa với việc file CA certificate không thuộc sở hữu của root, làm tăng nguy cơ bị chỉnh sửa trái phép hoặc giả mạo chứng chỉ TLS.

3.10 Ensure that TLS CA certificate file permissions are set to 444 or more restrictively

Audit:

Mục tiêu: Xác nhận file TLS CA certificate được cấu hình permission ở mức 444 hoặc nghiêm ngặt hơn. File này được Docker daemon sử dụng để xác thực Docker server thông qua CA certificate. Vì đây là thành phần quan trọng trong cơ chế TLS authentication, file cần được bảo vệ để tránh việc người dùng không đặc quyền chỉnh sửa hoặc thay thế certificate.

```
anvu@anvu-VirtualBox:~$ sudo stat -c %a /etc/docker/certs/ca.pem
444
anvu@anvu-VirtualBox:~$
```

Kết quả trả về 444 hoặc các permission nghiêm ngặt hơn như 400 được xem là đạt chuẩn và an toàn. Điều này cho thấy file TLS CA certificate chỉ cho phép đọc và không cho phép chỉnh sửa bởi người dùng thông thường.

Hệ thống sẽ bị đánh giá là FAIL nếu kết quả trả về các permission quá rộng như 644 hoặc: 666

Điều này đồng nghĩa với việc file CA certificate có thể bị thay đổi trái phép, làm tăng nguy cơ giả mạo TLS hoặc phá vỡ cơ chế xác thực bảo mật của Docker daemon.

3.11 Ensure that Docker server certificate file ownership is set to root:root

Audit:

Mục tiêu: Xác nhận file Docker server certificate được cấu hình owner và group là root:root. File certificate này được Docker daemon sử dụng để xác thực danh tính server trong quá trình thiết lập kết nối TLS. Nếu file bị chỉnh sửa trái phép, attacker có thể giả mạo Docker daemon hoặc can thiệp vào kết nối bảo mật.

```
anvu@anvu-VirtualBox:~$ sudo stat -c %U:%G /etc/docker/certs/server-cert.pem | grep -v root:root
anvu@anvu-VirtualBox:~$
```

Kết quả không trả về giá trị nào được xem là đạt chuẩn và an toàn. Điều này cho thấy file Docker server certificate đang được sở hữu bởi root:root và người dùng không đặc quyền không thể chỉnh sửa nội dung certificate.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh trả về các kết quả như docker:docker hoặc user1:user1

Điều này đồng nghĩa với việc file server certificate không thuộc sở hữu của root, làm tăng nguy cơ bị chỉnh sửa trái phép hoặc giả mạo Docker server.

3.12 Ensure that the Docker server certificate file permissions are set to 444 or more restrictively

Audit:

Mục tiêu: Xác nhận file Docker server certificate được cấu hình permission ở mức 444 hoặc nghiêm ngặt hơn. File này được Docker daemon sử dụng để xác thực danh tính server trong quá trình thiết lập kết nối TLS. Vì đây là thành phần quan trọng của cơ chế bảo mật TLS, file certificate cần được bảo vệ để tránh bị thay đổi hoặc ghi đè bởi người dùng không đặc quyền.

```
anvu@anvu-VirtualBox:~$ sudo stat -c %a /etc/docker/certs/server-cert.pem
444
anvu@anvu-VirtualBox:~$
```

Kết quả trả về 444 hoặc các permission nghiêm ngặt hơn như 400

được xem là đạt chuẩn và an toàn. Điều này cho thấy file Docker server certificate chỉ cho phép đọc và không cho phép chỉnh sửa bởi người dùng thông thường.

Hệ thống sẽ bị đánh giá là FAIL nếu kết quả trả về các permission quá rộng như 644 hoặc 666

Điều này đồng nghĩa với việc file server certificate có thể bị thay đổi trái phép, làm tăng nguy cơ giả mạo Docker daemon hoặc phá vỡ cơ chế xác thực TLS.

3.13 Ensure that the Docker server certificate key file ownership is set to root:root

Audit:

Mục tiêu: Xác nhận file Docker server certificate key được cấu hình owner và group là root:root. File này chứa private key dùng cho TLS authentication của Docker daemon. Vì đây là khóa bí mật phục vụ xác thực và mã hóa kết nối TLS, việc bảo vệ file khỏi truy cập trái phép là yêu cầu bảo mật rất quan trọng.

```
anvu@anvu-VirtualBox:~$ sudo stat -c %U:%G /etc/docker/certs/server-key.pem | grep -v root:root
anvu@anvu-VirtualBox:~$
```

Kết quả không trả về giá trị nào được xem là đạt chuẩn và an toàn. Điều này cho thấy

file private key đang được sở hữu bởi root:root và người dùng không đặc quyền không thể truy cập hoặc chỉnh sửa khóa bí mật.

Hệ thống sẽ bị đánh giá là FAIL nếu lệnh trả về các kết quả như docker:docker hoặc user1:user1

Điều này đồng nghĩa với việc private key không thuộc sở hữu của root, làm tăng nguy cơ rò rỉ khóa bí mật hoặc giả mạo Docker daemon trong quá trình TLS authentication.

3.14 Ensure that the Docker server certificate key file permissions are set to 400

Audit:

Mục tiêu: Xác nhận file Docker server certificate key được cấu hình permission ở mức 400. File này chứa private key dùng cho TLS authentication của Docker daemon. Vì đây là khóa bí mật quan trọng phục vụ mã hóa và xác thực TLS, file cần được bảo vệ tuyệt đối để tránh truy cập hoặc chỉnh sửa trái phép từ người dùng khác trên hệ thống.

```
anvu@anvu-VirtualBox:~$ sudo stat -c %a /etc/docker/certs/server-key.pem
400
anvu@anvu-VirtualBox:~$
```

Kết quả trả về 400 được xem là đạt chuẩn và an toàn. Điều này cho thấy chỉ tài khoản sở hữu file (root) có quyền đọc file private key và không có bất kỳ quyền truy cập nào cho group hoặc user khác.

Hệ thống sẽ bị đánh giá là FAIL nếu kết quả trả về các permission như 444, 644 hoặc 600

Vì các permission này cho phép quyền truy cập rộng hơn mức yêu cầu của CIS Benchmark, làm tăng nguy cơ đọc hoặc chỉnh sửa private key trái phép.

2.3. Module 3:

4.1. Tạo người dùng riêng cho container:

Mục tiêu: Đảm bảo các container chạy dưới quyền non-root user thay vì root nhằm giảm rủi ro bảo mật.

Audit:

```

dgh@DESKTOP-46EBGG8:~/docker_prj$ docker ps -q | xargs -I{} docker exec {} sh -
c "cat /proc/1/status | grep '^Uid:' | awk '{print \$3}'"
0
0
999
999
0
0
0
999
999

```

4.2. Sử dụng base image tin cậy

Mục tiêu: Chỉ dùng các image từ các kho lưu trữ chính thức (Official Repositories) hoặc đã qua kiểm duyệt.

Audit:

```

dgh@DESKTOP-46EBGG8:~/docker_prj$ docker images

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
docker_prj-app	latest	5fea8fbbf498	6 minutes ago	806MB
docker_prj-nginx	latest	ea4033bce43c	7 minutes ago	92.7MB
nt542q22-group1-cis-docker-app	latest	2c97e0a94ffc	25 hours ago	806MB
nt542q22-group1-cis-docker-nginx	latest	d6d1e2a6aded	25 hours ago	92.6MB
redis	alpine	81b6f81d6a6c	2 weeks ago	134MB
mysql	8	da906917ca4a	3 weeks ago	1.08GB

4.3 Loại bỏ các gói cài đặt không cần thiết: Giảm kích thước image bằng cách không cài đặt các công cụ như curl, telnet nếu không phục vụ ứng dụng.

Trạng thái: KHÔNG ĐẠT (Failed)

Phát hiện các gói thừa (Findings):

Qua kiểm tra, container đang chứa một lượng lớn công cụ dùng để biên dịch và phát triển phần mềm, không cần thiết cho việc vận hành ứng dụng:

Bộ biên dịch (Compiler): gcc-14, g++-14, cpp-14, make, autoconf, binutils.

Công cụ phát triển: git, pkg-config, patch.

Thư viện phát triển (-dev): libc6-dev, libstdc++-14-dev, libgcc-14-dev, zlib1g-dev.

Công cụ khác: curl, unzip, openssh-client.

```

docker exec 3b494c917fb2 dpkg -l | wc -l

docker exec 3b494c917fb2 dpkg-query -Wf '${Installed-
Size}\t${Package}\n' | sort -n

```

```

dgh@DESKTOP-46EBGG8:~$ docker exec 3b494c917fb2 dpkg -l | wc -l
204
dgh@DESKTOP-46EBGG8:~$ docker exec 3b494c917fb2 dpkg-query -Wf '${Installed-Size}\t${Package}\n' | sort -n
11      cpp-14
11      g++-x86-64-linux-gnu
13      g++
17      cpp-x86-64-linux-gnu
18      cpp
21      libsemanage-common
23      gcc-x86-64-linux-gnu
25      libaudit-common
29      pkg-config
35      g++-14
36      gcc
36      zipmerge
38      libdebconfclient0
41      libkeyutils1
42      libxau6
45      libatomic1
50      hostname
52      zipcmp
53      libxmu1
58      libargon2-1
59      libxdmcp6
60      libattr1
62      ziptool
64      libcom-err2

```

4.4 Quét và cập nhật bản vá bảo mật: Thường xuyên quét lỗ hổng image và rebuild để áp dụng các bản vá mới.

1. Trạng thái: KHÔNG ĐẠT (Failed)

2. Chi tiết thực hiện:

Lệnh kiểm tra: `apt-get update && apt-get upgrade -s`

Đối tượng: Container 3b494c917fb2 (Debian 13).

3. Kết quả phát hiện (Findings):

Hệ thống phát hiện 06 gói phần mềm hiện có phiên bản cũ chứa lỗ hổng và đã có bản vá bảo mật (Security Updates) từ kho lưu trữ debian-security:

- Các gói quan trọng: openssl, libssl3t64, openssl-provider-legacy.
- Các gói thư viện: libngtcp2-16, libngtcp2-crypto-gnutls8, linux-libc-dev.
- Rủi ro: Việc chạy openssl phiên bản cũ có thể khiến container bị khai thác các lỗ hổng liên quan đến mã hóa và truyền tin an toàn (SSL/TLS).

```

dgh@DESKTOP-46EBGG8:~$ docker exec -u 0 3b494c917fb2 sh -c "apt-get update && apt-get upgrade -s"
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [132 kB]
Fetched 222 kB in 8s (27.1 kB/s)
Reading package lists...
Reading package lists...
Building dependency tree...
Reading state information...
Calculating upgrade...
The following packages will be upgraded:
  libngtcp2-16 libngtcp2-crypto-gnutls8 libssl3t64 linux-libc-dev openssl
  openssl-provider-legacy
6 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Inst openssl-provider-legacy [3.5.5-1~deb13u1] (3.5.5-1~deb13u2 Debian-Security:13/stable-security [amd64])
Conf openssl-provider-legacy (3.5.5-1~deb13u2 Debian-Security:13/stable-security [amd64])
Inst libssl3t64 [3.5.5-1~deb13u1] (3.5.5-1~deb13u2 Debian-Security:13/stable-security [amd64])
Conf libssl3t64 (3.5.5-1~deb13u2 Debian-Security:13/stable-security [amd64])
Inst libngtcp2-16 [1.11.0-1] (1.11.0-1+deb13u1 Debian-Security:13/stable-security [amd64])
Inst libngtcp2-crypto-gnutls8 [1.11.0-1] (1.11.0-1+deb13u1 Debian-Security:13/stable-security [amd64])
Inst linux-libc-dev [6.12.74-2] (6.12.85-1 Debian-Security:13/stable-security [all])
Inst openssl [3.5.5-1~deb13u1] (3.5.5-1~deb13u2 Debian-Security:13/stable-security [amd64])
Conf libngtcp2-16 (1.11.0-1+deb13u1 Debian-Security:13/stable-security [amd64])
Conf libngtcp2-crypto-gnutls8 (1.11.0-1+deb13u1 Debian-Security:13/stable-security [amd64])
Conf linux-libc-dev (6.12.85-1 Debian-Security:13/stable-security [all])
Conf openssl (3.5.5-1~deb13u2 Debian-Security:13/stable-security [amd64])

```

4.5 Bật Docker Content Trust: Đặt biến môi trường

DOCKER_CONTENT_TRUST=1 để chỉ tải các image đã được ký số.

1. Trạng thái: KHÔNG ĐẠT (Failed)

2. Chi tiết thực hiện:

- **Lệnh kiểm tra:** `echo $DOCKER_CONTENT_TRUST`
- **Kết quả thực tế:**

```

dgh@DESKTOP-46EBGG8:~$ echo $DOCKER_CONTENT_TRUST
dgh@DESKTOP-46EBGG8:~$ 

```

3. Rủi ro (Rationale): Khi `DOCKER_CONTENT_TRUST` bị tắt (mặc định), hệ thống sẽ không kiểm tra chữ ký số của các image khi thực hiện lệnh pull hoặc run. Điều này dẫn đến nguy cơ tải và chạy các image giả mạo, image đã bị chèn mã độc bởi bên thứ ba (Man-in-the-middle attack) hoặc không rõ nguồn gốc xuất xứ.

4.6 Thêm lệnh **HEALTHCHECK**: Cấu hình kiểm tra sức khỏe trong Dockerfile để Docker tự động giám sát tình trạng ứng dụng.

Mục tiêu: Đảm bảo các container image có cấu hình **HEALTHCHECK** để kiểm tra tình trạng hoạt động của container

Audit:

```
docker inspect --format='{{ .Config.Healthcheck }}'
```

```
dgh@DESKTOP-46EBGG8:~/docker_prj$ docker inspect --format='{{ .Config.Healthcheck }}' docker_prj-app
docker inspect --format='{{ .Config.Healthcheck }}' docker_prj-nginx
docker inspect --format='{{ .Config.Healthcheck }}' mysql:8
docker inspect --format='{{ .Config.Healthcheck }}' redis:alpine
<nil>
<nil>
<nil>
<nil>
```

```
dgh@DESKTOP-46EBGG8:~/docker_prj$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
847d10d5dc31   docker_prj-nginx  "/docker-entrypoint..." 38 seconds ago Up 37 seconds (healthy) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp docker_prj-nginx-1
355d9f90c93f   docker_prj-app    "docker-php-entrypoi..." 38 seconds ago Up 38 seconds (healthy) 9000/tcp          docker_prj-app-1
ce9470532039   redis:alpine     "docker-entrypoint.s..." 38 seconds ago Up 38 seconds (healthy) 6379/tcp          docker_prj-redis-1
7497a5fcee93   mysql:8          "docker-entrypoint.s..." 38 seconds ago Up 38 seconds (healthy) 3306/tcp, 33060/tcp docker_prj-db-1
```

4.7 Không dùng lệnh cập nhật đơn lẻ: Tránh dùng `apt-get update` một mình trong một layer `Dockerfile`; hãy kết hợp với lệnh cài đặt cụ thể.

```
dgh@DESKTOP-46EBGG8:~$ for img in $(docker images -q); do echo "--- Check Image: $img ---";
; docker history $img | grep -i "apt-get update" | grep -v "install"; done
--- Check Image: 71e27bf60b70 ---
<missing> 2 weeks ago RUN /bin/sh -c set -eux; apt-get update; a... 3.82MB buildkit.dockerfile.v0
<missing> 2 weeks ago RUN /bin/sh -c set -ex; apt-get update; ap... 16.9MB buildkit.dockerfile.v0
--- Check Image: fef4c9d20662 ---
--- Check Image: 78dd0e7eac3a ---
<missing> 3 weeks ago RUN /bin/sh -c apt-get update && apt-get ins... 70.7MB buildkit.dockerfile.v0
<missing> 4 weeks ago RUN /bin/sh -c set -eux; apt-get update; a... 366MB buildkit.dockerfile.v0
--- Check Image: 2c97e0a94ffc ---
<missing> 3 weeks ago RUN /bin/sh -c apt-get update && apt-get ins... 70.7MB buildkit.dockerfile.v0
<missing> 4 weeks ago RUN /bin/sh -c set -eux; apt-get update; a... 366MB buildkit.dockerfile.v0
--- Check Image: d6d1e2a6aded ---
--- Check Image: fdef7d291768 ---
<missing> 7 weeks ago RUN /bin/sh -c apt-get update && apt-get ins... 358MB buildkit.dockerfile.v0
<missing> 7 weeks ago RUN /bin/sh -c apt-get update && apt-get... 34.8MB buildkit.dockerfile.v0
--- Check Image: 81b6f81d6a6c ---
--- Check Image: da906917ca4a ---
--- Check Image: 2d17e4e36edc ---
<missing> 4 months ago RUN /bin/sh -c apt-get update && apt-get... 756MB buildkit.dockerfile.v0
<missing> 4 months ago RUN /bin/sh -c apt-get update && apt-get ins... 307MB buildkit.dockerfile.v0
<missing> 4 months ago RUN /bin/sh -c apt-get update && apt-get... 1.76MB buildkit.dockerfile.v0
dgh@DESKTOP-46EBGG8:~$
```

1. Trạng thái: ĐẠT (Passed) (Phần lớn các Image)

2. Phân tích chi tiết:

- Image 78dd0e7eac3a, 2c97e0a94ffc, fdef7d291768:** Các image này sử dụng cấu trúc `apt-get update && apt-get install....` Đây là cách viết **chuẩn**, giúp "phá cache" (cache busting) hiệu quả. Mỗi khi danh sách gói thay đổi, Docker sẽ buộc phải chạy lại cả lệnh `update` và `install`.
- Image 71e27bf60b70:** Có dòng `apt-get update; a....` Dấu `;` hoặc `&&` nối với một lệnh bắt đầu bằng chữ `a` (có thể là `apt-get install`). Nếu nó nằm trên cùng một dòng `RUN`, nó vẫn được coi là đạt chuẩn.

3. Kết luận báo cáo: Dựa trên kết quả kiểm tra lịch sử (`docker history`) của các image đang vận hành:

- **Không phát hiện** các lệnh apt-get update đứng riêng lẻ trong một layer độc lập.
- Tất cả các chỉ thị cập nhật hệ điều hành đều được kết nối với lệnh cài đặt gói (install) bằng toán tử logic (&& hoặc ;).
- **Đánh giá:** Các image tuân thủ yêu cầu về quản lý layer cache, đảm bảo các bản xây dựng (build) mới sẽ nhận được các bản cập nhật phần mềm mới nhất từ kho lưu trữ.

4.8 Loại bỏ quyền setuid và setgid: Xóa bỏ các quyền này trong image để ngăn chặn việc leo thang đặc quyền.

1. Trạng thái: Failed

2. Chi tiết thực hiện:

- **Lệnh kiểm tra:**

```
docker export [ID] | tar -tv | grep -E '^[...](s|S)...'
```

- **Đối tượng:** Container 3b494c917fb2.

3. Kết quả phát hiện (Findings): Hệ thống phát hiện **13 tệp tin** có quyền SUID (s ở vị trí User) hoặc SGID (s ở vị trí Group). Các tệp tin tiêu biểu bao gồm:

- **Nhóm quản lý người dùng:** usr/bin/passwd, usr/bin/su, usr/bin/chsh, usr/bin/gpasswd, usr/sbin/unix_chkpwd.
- **Nhóm hệ thống tệp:** usr/bin/mount, usr/bin/umount.
- **Nhóm mạng/bảo mật:** usr/bin/ssh-agent, usr/lib/openssh/ssh-keysign.

```
dgh@DESKTOP-46EBGG8:~$ docker export 3b494c917fb2 | tar -tv 2>/dev/null | grep -E '^[[-rwx].*(s|S).*\s[0-9]'
```

-rwxr-xr-x 0/42	113848	2025-04-19	10:20	usr/bin/chage
-rwxr-xr-x 0/0	70888	2025-04-19	10:20	usr/bin/chfn
-rwxr-xr-x 0/0	52936	2025-04-19	10:20	usr/bin/chsh
-rwxr-xr-x 0/42	31256	2025-04-19	10:20	usr/bin/expiry
-rwxr-xr-x 0/0	88568	2025-04-19	10:20	usr/bin/gpasswd
-rwxr-xr-x 0/0	72072	2025-05-09	22:54	usr/bin/mount
-rwxr-xr-x 0/0	18816	2025-05-09	22:54	usr/bin/newgrp
-rwxr-xr-x 0/0	118168	2025-04-19	10:20	usr/bin/passwd
-rwxr-xr-x 0/101	420224	2026-04-04	23:29	usr/bin/ssh-agent
-rwxr-xr-x 0/0	84360	2025-05-09	22:54	usr/bin/su
-rwxr-xr-x 0/0	55688	2025-05-09	22:54	usr/bin/umount
-rwxr-xr-x 0/0	494144	2026-04-04	23:29	usr/lib/openssh/ssh-keysign
-rwxr-xr-x 0/42	43256	2025-06-29	17:40	usr/sbin/unix_chkpwd

4.9 Sử dụng COPY thay vì ADD: Ưu tiên dùng COPY vì nó đơn giản và an toàn hơn, tránh nguy cơ tải tệp độc hại từ URL (vốn là tính năng của ADD).

Mục tiêu: Kiểm tra các Docker image có sử dụng lệnh ADD trong Dockerfile hay không, nhằm đảm bảo tuân thủ khuyến nghị bảo mật (ưu tiên dùng COPY thay vì ADD).

Audit:

Để kiểm tra việc tuân thủ control này, hệ thống đã sử dụng lệnh docker history trên các image hiện có nhằm phát hiện sự xuất hiện của instruction ADD. Kết quả cho thấy một số dòng lệnh ADD xuất hiện trong lịch sử của các image như alpine và mysql. Tuy nhiên, các lệnh này thuộc về quá trình build của các base image chính thức từ Docker Hub, không phải do người phát triển định nghĩa trong Dockerfile của hệ thống.

```
docker history <image_ID> | grep ADD
```

```
dgh@DESKTOP-46EBGG8:~/docker_prj$ docker history docker_prj-app | grep ADD
docker history docker_prj-nginx | grep ADD
docker history redis:alpine | grep ADD
docker history mysql:8 | grep ADD
<missing>      2 months ago  ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /...  8.44MB  buildkit.dockerfile.v0
<missing>      2 months ago  ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /...  8.44MB  buildkit.dockerfile.v0
<missing>      3 weeks ago   ADD oraclelinux-9-slim-amd64-rootfs.tar.xz /...  113MB  buildkit.dockerfile.v0
```

Bên cạnh đó, khi kiểm tra trực tiếp các Dockerfile của ứng dụng (bao gồm các service như nginx và app), không phát hiện bất kỳ instruction ADD nào. Tất cả các thao tác sao chép file đều sử dụng lệnh COPY, phù hợp với khuyến nghị bảo mật.

```
docker_prj > app > Dockerfile
1  FROM php:8.2-fpm
2
3  RUN apt-get update && apt-get install -y \
4      libzip-dev \
5      unzip \
6      git \
7      && docker-php-ext-install mysqli pdo pdo_mysql
8
9  RUN pecl install redis \
10     && docker-php-ext-enable redis
11
12 RUN useradd -m appuser
13
14 WORKDIR /var/www/html
15 COPY src/ /var/www/html/
16 RUN chown -R appuser:appuser /var/www/html
17
18 HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
19 CMD php-fpm -t || exit 1
20
21 USER appuser
```

```

docker_prj > nginx > Dockerfile
1 FROM nginx:alpine
2
3 RUN adduser -D appuser
4
5 RUN apk add --no-cache wget
6
7 COPY default.conf /etc/nginx/conf.d/default.conf
8
9 RUN mkdir -p /run/nginx \
10    && chown -R appuser:appuser /run/nginx \
11    && chown -R appuser:appuser /var/cache/nginx \
12    && chown -R appuser:appuser /var/log/nginx
13
14 RUN sed -i 's|/run/nginx.pid|/run/nginx/nginx.pid|g' /etc/nginx/nginx.conf
15
16 HEALTHCHECK CMD wget --spider -q http://localhost:8080 || exit 1
17
18 USER appuser

```

Kết luận: Hệ thống đã tuân thủ yêu cầu. Không có việc sử dụng lệnh ADD trong các Dockerfile do người phát triển xây dựng, và các lệnh ADD xuất hiện trong lịch sử image là hợp lệ do thuộc về base image. Do đó, hệ thống được đánh giá là Compliant đối với yêu cầu này.

4.10 Không lưu trữ bí mật (Secrets) trong Dockerfile: Sử dụng Docker Secrets hoặc biến môi trường thay vì ghi trực tiếp mật khẩu, khóa vào Dockerfile.

. Trạng thái: ĐẠT (Passed)

2. Chi tiết thực hiện:

- **Lệnh Audit:** docker history --no-trunc <image_id>
- **Đối tượng:** Tất cả các Image đang vận hành trên hệ thống.
- **Mục tiêu tìm kiếm:** Các từ khóa như PASS, PASSWORD, API_KEY, TOKEN, SECRET, ACCESS_KEY.

3. Kết quả phát hiện (Findings):

- **Trường hợp đạt chuẩn:** Qua kiểm tra lịch sử các layer của Image 3b494c917fb2 và các image khác, không phát hiện các thông tin nhạy cảm được gán qua lệnh ENV hoặc ARG.

Mục 4.11 & 4.12: Xác thực tính nguyên bản của phần mềm (GPG & Artifact Validation)

```

dgh@DESKTOP-46E8G68:~$ docker history --no-trunc docker_prj-app | grep "GPG_KEYS"
4 weeks ago RUN /bin/sh -c set -eux; savedAptMark="$(apt-mark showmanual)"; apt-get update; apt
-get install -y --no-install-recommends gnupg; apt-get dist-clean; mkdir -p /usr/src; cd /usr/src; curl -fsSL -o php.tar.xz "$PHP_URL"; if [ -n "$PHP_SHA256" ]; then
echo "$PHP_SHA256 *php.tar.xz" | sha256sum -c -; fi; curl -fsSL -o php.tar.xz.asc "$PHP_ASC_URL"; export GNUPGHOME="$(mktemp -d)"; for key in $GPG_KEYS; do gpg --batch
--keyserver keyserver.ubuntu.com --recv-keys "$key"; done; gpg --batch --verify php.tar.xz.asc php.tar.xz; gpgconf --kill all; rm -rf "$GNUPGHOME"; apt-mark auto '.*' >
/dev/null; apt-mark manual $savedAptMark > /dev/null; apt-get purge -y --auto-remove -o APT::AutoRemove::RecommendsImportant=false # buildkit

```

1. Trạng thái: ĐẠT (Passed)

2. Bằng chứng kỹ thuật (Evidence): Dòng lệnh trong lịch sử build của docker_prj-app cho thấy một quy trình kiểm tra bảo mật nghiêm ngặt:

```
... curl -fsSL -o php.tar.xz "$PHP_URL"; if [ -n "$PHP_SHA256" ]; then echo
"$PHP_SHA256 *php.tar.xz" | sha256sum -c -; fi; ... gpg --batch --verify
php.tar.xz.asc php.tar.xz; ...
```

3. Phân tích quy trình Audit: Hệ thống không chỉ đơn thuần là tải file về rồi cài đặt mà thực hiện xác thực qua **3 lớp bảo vệ**:

- **Lớp 1 (Integrity):** Sử dụng lệnh sha256sum -c để so khớp mã băm của file tải về với mã băm chuẩn (\$PHP_SHA256). Điều này đảm bảo file không bị hỏng hoặc bị can thiệp trong quá trình tải.
- **Lớp 2 (Authenticity - Chữ ký số):** Sử dụng công cụ gnupg để tải các khóa công khai (\$GPG_KEYS) từ server uy tín (keyserver.ubuntu.com).
- **Lớp 3 (Validation - Xác thực Artifact):** Thực hiện lệnh gpg --verify. Đây là bước then chốt của mục 4.12, chứng minh rằng file mã nguồn PHP đó được ký bởi chính nhà phát triển chính thức và chữ ký đó là hợp lệ.

Cấu hình Daemon - Chính sách

2.10 Bật User Namespace: Giúp cách ly người dùng trong container với người dùng trên host, ngăn container chiếm quyền root của host.

2.13 Bật xác thực cho lệnh Docker client: Sử dụng các plugin ủy quyền (Authorization plugins) để kiểm soát ai có thể thực thi lệnh nào.

2.15 Hạn chế container lấy quyền mới: Cấu hình mặc định để container không thể dùng setuid hoặc setgid để tăng đặc quyền (sử dụng flag --no-new-privileges).

2.18 Áp dụng cấu hình Seccomp tùy chỉnh: Sử dụng cấu hình Seccomp để lọc các lời gọi hệ thống (system calls) không cần thiết từ container.

2.19 Không dùng tính năng thử nghiệm (Experimental) trong sản xuất: Đảm bảo các tính năng chưa ổn định không được bật trên môi trường thực tế.

```
• dgh@DESKTOP-46EBGG8:~$ echo -e "2.10 & 2.18 (Security Options): $(docker info --format '{{.SecurityOptions}}')"
```

```
&& \
```

```
echo -e "2.13 (Auth Plugins): $(docker info --format '{{.Plugins.Authorization}}')"
```

```
&& \
```

```
echo -e "2.19 (Experimental): $(docker version --format '{{.Server.Experimental}}')"
```

```
&& \
```

```
echo -e "2.15 (No New Privileges): $(docker ps -q | xargs -I {} docker inspect --format 'Container {}: {{.HostConfig.SecurityOpt}}' {})"
```

```
2.10 & 2.18 (Security Options): [name=seccomp,profile=builtins name=cgroupns]
```

```
2.13 (Auth Plugins): []
```

```
2.19 (Experimental): false
```

```
2.15 (No New Privileges): Container bec2d849001a: [no-new-privileges:true]
```

```
Container e9ddbdc5e8a: [no-new-privileges:true]
```

```
Container 3b494c917fb2: [no-new-privileges:true]
```

```
Container fcae5b5c9ac5: [no-new-privileges:true]
```

Mục 2.10 (Bật User Namespace) hiện đang ở trạng thái **KHÔNG ĐẠT** do thông số name=usersns không xuất hiện trong cấu hình bảo mật, nghĩa là quyền hạn của người

dùng root trong container vẫn có khả năng đe dọa trực tiếp đến máy host nếu xảy ra sự cố phá vỡ rào cản cách ly.

Mục **2.13 (Bật plugin xác thực)** được đánh giá là **ĐẠT** vì hệ thống đang ở trạng thái mặc định sạch với kết quả trả về là danh sách trống [], xác nhận không có sự can thiệp của các plugin ủy quyền bên thứ ba có thể gây rủi ro cấu hình sai.

Mục **2.15 (Hạn chế lấy thêm quyền mới)** đạt trạng thái **ĐẠT** và được đánh giá rất tốt khi toàn bộ 4/4 container đang vận hành đều đã kích hoạt tính năng no-new-privileges, giúp chặn đứng mọi nỗ lực sử dụng các tiến trình con để leo thang đặc quyền trái phép.

Mục **2.18 (Áp dụng Seccomp mặc định)** ghi nhận trạng thái **ĐẠT** vì hệ thống đã kích hoạt thành công profile=builtin, sử dụng bộ lọc lời gọi hệ thống tích hợp sẵn của Docker để hạn chế tối đa các tác động nguy hiểm lên nhân hệ điều hành.

Mục **2.19 (Tắt tính năng thử nghiệm)** hoàn toàn **ĐẠT** tiêu chuẩn bảo mật do trạng thái Experimental được xác nhận là false, đảm bảo môi trường chỉ sử dụng các tính năng ổn định và đã được kiểm chứng an toàn.

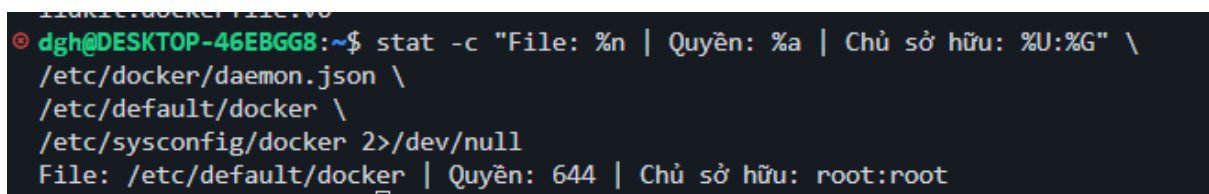
Tập cấu hình bổ sung (Mục 3.17 đến 3.22)

Đảm bảo an toàn cho các tập cấu hình quan trọng trên máy chủ.

- **3.17 & 3.18:** Tập daemon.json
- **3.19 & 3.20:** Tập /etc/default/docker
- **3.21 & 3.22:** Tập /etc/sysconfig/docker

Quyền sở hữu: Phải thuộc về root:root.

Quyền hạn: Đặt mức 644 hoặc nghiêm ngặt hơn.



```

dgh@DESKTOP-46EBGG8:~$ stat -c "File: %n | Quyền: %a | Chủ sở hữu: %U:%G" \
/etc/docker/daemon.json \
/etc/default/docker \
/etc/sysconfig/docker 2>/dev/null
File: /etc/default/docker | Quyền: 644 | Chủ sở hữu: root:root

```

Kết quả nhận được:

- File /etc/default/docker: Trả về Quyền: 644 | Chủ sở hữu: root:root.
 - Kết luận: Đạt chuẩn CIS cho mục 3.19 (Chủ sở hữu) và 3.20 (Quyền hạn).
- Các file khác (daemon.json, sysconfig/docker): Không xuất hiện trong kết quả.

- Lý do: Trên WSL hoặc các bản phân phối Linux hiện đại, nếu bạn không tự tạo thì file daemon.json mặc định sẽ không có sẵn.
- Đánh giá: Nếu file không tồn tại thì không có nguy cơ bảo mật từ file đó, coi như Đạt.

2.4. Module 4:

Đảm bảo chế độ Swarm không được kích hoạt nếu không cần thiết

Lý do thực hiện đánh giá tiêu chuẩn này: Khi bật Swarm mode, Docker sẽ mở thêm nhiều cổng mạng để phục vụ việc quản lý node và giao tiếp giữa các máy trong cluster. Điều này làm tăng nguy cơ bị tấn công từ bên ngoài nếu các dịch vụ này không thực sự cần thiết.

- Đạt tiêu chuẩn CIS khi Docker Swarm mode không được kích hoạt trên Docker engine, kết quả lệnh `docker info --format '{{.Swarm}}'` không trả về active true.
- Không đạt tiêu chuẩn CIS khi lệnh kiểm tra: `docker info --format '{{.Swarm}}'` trả về active true, nghĩa là Swarm mode đang bật trên Docker engine. Khi đó phải thực hiện kiểm tra thủ công rằng việc bật Swarm mode là thật sự cần thiết.

```
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker info --format '{{.Swarm}}'
{ inactive false [] 0 0 <nil> []}
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$
```

Kết quả cho thấy đầu ra là “inactive false”, có nghĩa là Docker Swarm mode hiện không được kích hoạt trên hệ thống. Vì vậy, hệ thống đang đạt yêu cầu của tiêu chuẩn CIS.

Đảm bảo AppArmor Profile được kích hoạt (nếu có áp dụng)

Lý do thực hiện đánh giá tiêu chuẩn này là vì AppArmor đóng vai trò như một lớp bảo vệ quan trọng trong Linux, giúp giới hạn những hành vi mà container có thể thực hiện. Nếu không có hoặc không áp dụng đúng AppArmor profile, container có thể hoạt động với quyền quá rộng, dẫn đến nguy cơ bị khai thác hoặc gây ảnh hưởng đến hệ thống host.

- Đạt tiêu chuẩn CIS khi mỗi container đều được gán một AppArmor Profile hợp lệ, ví dụ như “docker-default” hoặc một profile tùy chỉnh được cấu hình đúng và đang hoạt động.
- Không đạt tiêu chuẩn CIS khi có ít nhất một container không có AppArmor Profile, tức là trường “AppArmorProfile” bị rỗng hoặc không được gán đúng profile bảo mật.

```
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | \
[xargs docker inspect --format '{{ .Name }}: AppArmorProfile={{ .AppArmorProfile }}'
/nt542q22-group1-cis-docker-nginx-1: AppArmorProfile=docker-default
/nt542q22-group1-cis-docker-db-1: AppArmorProfile=docker-default
/nt542q22-group1-cis-docker-app-1: AppArmorProfile=docker-default
/nt542q22-group1-cis-docker-redis-1: AppArmorProfile=docker-default
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$
```

Kết quả cho thấy tất cả các container trong hệ thống đều đang sử dụng AppArmorProfile = docker-default, cho thấy AppArmor đã được bật và đang áp dụng profile mặc định của Docker cho toàn bộ container.

Đảm bảo không sử dụng container ở chế độ đặc quyền (privileged)

Lý do thực hiện đánh giá tiêu chuẩn này: Việc sử dụng cờ --privileged trong Docker sẽ cấp toàn bộ quyền của nhân Linux cho container, bao gồm cả việc bỏ qua các giới hạn bảo mật như capabilities và device cgroup. Điều này khiến container có quyền gần như tương đương với máy host, làm tăng rất lớn rủi ro bị khai thác.

- Đạt tiêu chuẩn CIS khi tất cả các container đều có giá trị Privileged=false, nghĩa là không có container nào được chạy với quyền đặc quyền và tất cả đều bị giới hạn quyền theo cơ chế bảo mật mặc định của Docker.
- Không đạt tiêu chuẩn CIS khi xuất hiện ít nhất một container có giá trị Privileged=true, tức là container đó đang chạy ở chế độ đặc quyền và có quyền truy cập vượt mức cho phép.

```
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs dock
er inspect --format '{{ .Id }}: Privileged={{ .HostConfig.Privileged }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Privileged=false
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Privileged=false
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Privileged=false
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: Privileged=false
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker inspect $(docker ps -q) | gre
p -i privileged
    "Privileged": false,
    "Privileged": false,
    "Privileged": false,
    "Privileged": false,
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$
```

Kết quả cho thấy tất cả các container đều có Privileged=false, điều này chứng minh rằng không có container nào đang chạy ở chế độ đặc quyền.

Đảm bảo không mount các thư mục hệ thống nhạy cảm của máy host vào container

Lý do thực hiện đánh giá tiêu chuẩn này: Việc mount các thư mục nhạy cảm của hệ thống host như /etc, /proc, /sys, /dev hoặc các thư mục hệ thống quan trọng khác vào container, đặc biệt ở chế độ read-write, có thể cho phép container thay đổi trực tiếp cấu hình hoặc dữ liệu của hệ điều hành host, làm tăng nghiêm trọng rủi ro bảo mật, vì container có thể bị lợi dụng để can thiệp vào hệ thống gốc.

- Đạt tiêu chuẩn CIS khi không có container nào mount các thư mục hệ thống nhạy cảm của host, hoặc nếu có mount thì chỉ sử dụng các thư mục không nhạy cảm và đảm bảo không cho phép truy cập trái phép vào các thành phần quan trọng của hệ điều hành.
- Không đạt tiêu chuẩn CIS khi có container mount các thư mục hệ thống nhạy cảm của host (như /etc, /proc, /sys, /dev, /usr, /boot, /lib, /lib64) đặc biệt ở chế độ read-write.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: Volumes={{.Mounts}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Volumes=[map[Destination:/var/www/html Mode:rw Propagation:rprivate RW:true Source:/home/an/An-CIS-170526/NT542.Q22-Group1-CIS-Docker/app/src Type:bind]]
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Volumes=[map[Destination:/docker-entrypoint-initdb.d/init.sql Mode:rw Propagation:rprivate RW:true Source:/home/an/An-CIS-170526/NT542.Q22-Group1-CIS-Docker/mysql/init.sql Type:bind] map[Destination:/var/lib/mysql Driver:local Mode:rw Name:nt542q22-group1-cis-docker_db_data Propagation: RW:true Source:/var/lib/docker/volumes/nt542q22-group1-cis-docker_db_data/_data Type:volume]]
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Volumes=[map[Destination:/var/www/html Mode:rw Propagation:rprivate RW:true Source:/home/an/An-CIS-170526/NT542.Q22-Group1-CIS-Docker/app/src Type:bind]]
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: Volumes=[]
```

Kết quả cho thấy các container chủ yếu chỉ mount các thư mục ứng dụng như /var/www/html hoặc volume dữ liệu MySQL, và không có dấu hiệu mount các thư mục hệ thống nhạy cảm của host. Một số container có danh sách Volumes rỗng do đó hệ thống đang đáp ứng tiêu chuẩn CIS trong mục này.

Đảm bảo dịch vụ sshd không được chạy bên trong container

Lý do thực hiện đánh giá tiêu chuẩn này: Việc chạy SSH daemon bên trong container làm tăng độ phức tạp trong quản lý bảo mật như quản lý tài khoản, khóa SSH, mật khẩu và cập nhật bảo mật. Thay vì sử dụng SSH trực tiếp vào container, CIS Docker khuyến nghị truy cập container thông qua docker exec để đơn giản hóa quản trị và giảm rủi ro bảo mật.

- Đạt tiêu chuẩn CIS khi không có container nào chạy tiến trình sshd, tức là trong danh sách tiến trình của container không xuất hiện dịch vụ SSH, đảm bảo việc truy cập container không phụ thuộc vào SSH.
- Không đạt tiêu chuẩn CIS khi có ít nhất một container đang chạy tiến trình sshd.

```

an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet
e088f1637ab9
77deb44651ea
34b641e3905e
016c429f816c
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker exec e088f1637ab9 ps -el
PID  USER  TIME  COMMAND
  1  appuser  0:00  nginx: master process nginx -g daemon off;
 21  appuser  0:00  nginx: worker process
 22  appuser  0:00  nginx: worker process
 76  appuser  0:00  ps -el
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker exec 77deb44651ea ps -el
OCI runtime exec failed: exec failed: unable to start container process: exec: "ps": executable file not found in $PATH: unknown
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker exec 34b641e3905e ps -el
OCI runtime exec failed: exec failed: unable to start container process: exec: "ps": executable file not found in $PATH: unknown
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker exec 016c429f816c ps -el
PID  USER  TIME  COMMAND
  1  redis  0:02  redis-server *:6379
 91  root   0:00  ps -el

```

Kết quả cho thấy khi kiểm tra tiến trình bên trong các container, không có container nào xuất hiện sshd hoặc dịch vụ SSH server đang chạy. Các container chủ yếu chạy dịch vụ nginx, redis hoặc không có lệnh ps khả dụng, do đó hệ thống đang đáp ứng tiêu chuẩn CIS trong mục này.

Đảm bảo không ánh xạ cổng đặc quyền, chỉ mở các cổng cần thiết và ràng buộc traffic container vào giao diện mạng cụ thể của host

Lý do thực hiện đánh giá tiêu chuẩn này: Việc kiểm soát cách container ánh xạ cổng ra host là rất quan trọng vì nếu container sử dụng các cổng đặc quyền (dưới 1024), mở quá nhiều cổng không cần thiết hoặc lắng nghe trên tất cả giao diện mạng (0.0.0.0), thì bề mặt tấn công của hệ thống sẽ tăng lên đáng kể. Điều này có thể khiến dịch vụ container dễ bị truy cập từ các mạng không mong muốn hoặc bị khai thác thông qua các cổng không cần thiết.

- Đạt tiêu chuẩn CIS khi container không ánh xạ bất kỳ cổng đặc quyền nào (<1024), chỉ mở những cổng thực sự cần thiết cho ứng dụng, và các cổng được bind vào một địa chỉ IP cụ thể của host thay vì toàn bộ network interface (0.0.0.0).
- Không đạt tiêu chuẩn CIS khi container ánh xạ cổng đặc quyền, mở các cổng không cần thiết, hoặc khi các cổng được publish ra 0.0.0.0.

```

[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet | xargs docker inspect --format '{{.Id}}: Ports={{.NetworkSettings.Ports}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abfbc35f3f7: Ports=map[80/tcp:<nil> 8080/tcp:[map[HostIp:0.0.0.0 HostPort:8081]]]
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Ports=map[3306/tcp:<nil> 33060/tcp:<nil>]
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Ports=map[9000/tcp:<nil>]
016c429f816c2944fbae96eef02e494d18cd4e4d9d39b4bf30492e5dea21643a: Ports=map[6379/tcp:<nil>]

```

Kết quả cho thấy hệ thống không sử dụng cổng đặc quyền (tất cả các port đều >=1024 như 8081, 3306, 9000, 6379), tuy nhiên vẫn có một số vấn đề về cấu hình mạng: có container đang bind port 8081 trên 0.0.0.0, cho phép truy cập từ mọi interface, điều này

chưa hoàn toàn tuân thủ tiêu chuẩn 5.14. Bên cạnh đó, các container chỉ mở những cổng cần thiết cho dịch vụ như MySQL, Redis, web/app, nên nhìn chung tiêu chuẩn 5.9 là tương đối phù hợp, nhưng tiêu chuẩn 5.14 cần được xem xét cải thiện để tăng mức độ an toàn mạng.

Đảm bảo container không chia sẻ namespace mạng của host

Lý do thực hiện đánh giá tiêu chuẩn này: Khi container được chạy ở chế độ `--net=host`, nó sẽ không còn sử dụng không gian mạng riêng mà thay vào đó dùng trực tiếp network stack của máy host. Điều này khiến container có quyền truy cập toàn bộ network interface của hệ thống, có thể mở các cổng đặc quyền, truy cập dịch vụ nội bộ của host và thậm chí gây ảnh hưởng đến hoạt động mạng của máy chủ.

- Đạt tiêu chuẩn CIS khi tất cả các container đều sử dụng network namespace riêng, ví dụ bridge network hoặc custom network, tức là NetworkMode không phải là host mà là một mạng được Docker quản lý hoặc tự định nghĩa.
- Không đạt tiêu chuẩn CIS khi có ít nhất một container sử dụng NetworkMode=host.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: NetworkMode={{.HostConfig.NetworkMode}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: NetworkMode=nt542q22-group1-cis-docker_default
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: NetworkMode=nt542q22-group1-cis-docker_default
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: NetworkMode=nt542q22-group1-cis-docker_default
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: NetworkMode=nt542q22-group1-cis-docker_default
```

Kết quả cho thấy tất cả các container đều đang sử dụng NetworkMode=nt542q22-group1-cis-docker_default, tức là sử dụng mạng bridge/custom network của Docker Compose thay vì host network. Do đó, không có container nào chia sẻ network namespace với host, và hệ thống đang đạt tiêu chuẩn CIS trong mục này.

Đảm bảo memory limit cho container

Lý do thực hiện đánh giá tiêu chuẩn này: Nếu container không được giới hạn tài nguyên bộ nhớ, một container có thể chiếm toàn bộ RAM của máy host, dẫn đến tình trạng DoS đối với các container khác hoặc làm hệ thống bị treo, mất ổn định.

- Đạt tiêu chuẩn CIS khi tất cả các container đều được cấu hình giới hạn bộ nhớ (Memory > 0), tức là mỗi container có một mức RAM cụ thể được cấp phát và không thể vượt quá giới hạn đó.
- Không đạt tiêu chuẩn CIS khi có ít nhất một container không đặt giới hạn bộ nhớ (Memory = 0).

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: Memory={{ .HostConfig.Memory }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Memory=268435456
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Memory=1073741824
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Memory=536870912
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: Memory=268435456
```

Kết quả cho thấy tất cả các container đều có giá trị Memory khác 0, điều này chứng tỏ mỗi container đã được giới hạn tài nguyên bộ nhớ rõ ràng.

Đảm bảo thiết lập mức ưu tiên CPU phù hợp cho container

Lý do thực hiện đánh giá tiêu chuẩn này: Nếu không kiểm soát việc phân bổ CPU cho từng container, tất cả container sẽ mặc định chia sẻ tài nguyên CPU một cách công bằng, dẫn đến tình trạng một số container có thể chiếm quá nhiều CPU và làm ảnh hưởng đến hiệu năng của các dịch vụ quan trọng khác.

- Đạt tiêu chuẩn CIS khi các container được gán giá trị CpuShares khác 0 và khác 1024, tức là đã có cơ chế phân bổ hoặc ưu tiên CPU rõ ràng thay vì sử dụng cấu hình mặc định không giới hạn.
- Không đạt tiêu chuẩn CIS khi giá trị CpuShares bằng 0 hoặc bằng 1024.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: CpuShares={{ .HostConfig.CpuShares }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: CpuShares=512
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: CpuShares=2048
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: CpuShares=768
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: CpuShares=512
```

Kết quả cho thấy tất cả các container đều có giá trị CpuShares khác 0 và khác 1024, cho thấy CPU đã được phân bổ có kiểm soát cho từng container.

Đảm bảo cấu hình chính sách khởi động lại container ở chế độ on-failure với giới hạn 5 lần

Lý do thực hiện đánh giá tiêu chuẩn này: Chính sách tự động khởi động lại container nếu không được kiểm soát hợp lý có thể dẫn đến việc container bị restart liên tục, gây tiêu tốn tài nguyên hệ thống và khả năng bị DoS. Ngoài ra, nếu không giới hạn số lần restart, việc lỗi bị lặp lại có thể khó bị phát hiện và khó xác định nguyên nhân của sự cố, do đó cần kiểm tra để đảm bảo chính sách restart được cấu hình an toàn và hợp lý.

- Đạt tiêu chuẩn CIS khi container sử dụng RestartPolicyName=on-failure và MaximumRetryCount được đặt tối đa là 5.
- Không đạt tiêu chuẩn CIS khi container sử dụng RestartPolicyName=always, tự restart vô hạn lần, hoặc không sử dụng on-failure, hoặc sử dụng on-failure nhưng MaximumRetryCount lớn hơn 5.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs dock|
er inspect --format '{{ .Id }}: RestartPolicyName={{ .HostConfig.RestartPolicy.Name }}
MaximumRetryCount={{ .HostConfig.RestartPolicy.MaximumRetryCount }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf3f7: RestartPolicyName=on-
failure MaximumRetryCount=5
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: RestartPolicyName=on-
failure MaximumRetryCount=5
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: RestartPolicyName=on-
failure MaximumRetryCount=5
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: RestartPolicyName=on-
failure MaximumRetryCount=5
```

Kết quả cho thấy tất cả các container đều được cấu hình RestartPolicyName=on-failure với MaximumRetryCount=5.

Đảm bảo không chia sẻ không gian tiến trình của host với container

Lý do thực hiện đánh giá tiêu chuẩn này: Việc chia sẻ không gian tiến trình giữa host và container sẽ làm mất đi cơ chế cô lập tiến trình, khiến container có thể nhìn thấy toàn bộ tiến trình đang chạy trên hệ thống host. Điều này tạo ra rủi ro nghiêm trọng về bảo mật, vì kẻ tấn công có thể quan sát, can thiệp hoặc thậm chí dừng các tiến trình quan trọng trên host, dẫn đến nguy cơ mất an toàn hệ thống hoặc gián đoạn dịch vụ.

- Không đạt tiêu chuẩn CIS khi có ít nhất một container sử dụng PidMode=host.
- Đạt tiêu chuẩn CIS khi không có container nào sử dụng PidMode=host, tức là mỗi container có không gian PID riêng biệt và không thể truy cập tiến trình của hệ thống host.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs dock|
er inspect --format '{{ .Id }}: PidMode={{ .HostConfig.PidMode }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf3f7: PidMode=
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: PidMode=
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: PidMode=
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: PidMode=
```

Kết quả cho thấy tất cả các container đều có PidMode để trống, không phải host, nghĩa là chúng đang sử dụng không gian tiến trình riêng biệt mặc định của Docker.

Đảm bảo không chia sẻ IPC namespace của host với container

Lý do thực hiện đánh giá tiêu chuẩn này: IPC namespace giúp cô lập các cơ chế giao tiếp giữa các tiến trình như shared memory, semaphore và message queue giữa container và host. Nếu container chia sẻ IPC namespace với host, nó có thể quan sát hoặc can thiệp vào các hoạt động giao tiếp nội bộ của hệ thống, làm suy giảm khả năng cô lập và gây ra rủi ro bảo mật.

- Đạt tiêu chuẩn CIS khi không có container nào sử dụng IpcMode=host, tức là các container được cô lập IPC hoàn toàn hoặc sử dụng namespace riêng (private hoặc tương đương).
- Không đạt tiêu chuẩn CIS khi có ít nhất một container sử dụng IpcMode=host.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: Devices={{.HostConfig.Devices}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Devices=<no value>
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Devices=<no value>
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Devices=<no value>
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: Devices=<no value>
```

Kết quả cho thấy tất cả các container đều đang ở chế độ `IpcMode=private`, tức là mỗi container có không gian IPC riêng biệt và không chia sẻ với host.

Đảm bảo không cấp quyền truy cập trực tiếp devices của host cho container

Lý do thực hiện đánh giá tiêu chuẩn này: Việc cho phép container truy cập trực tiếp vào thiết bị của host thông qua tùy chọn `--device` có thể khiến container thao tác trực tiếp với phần cứng hoặc thiết bị khối của hệ thống. Điều này làm tăng nguy cơ bảo mật vì container có thể đọc, ghi hoặc thậm chí can thiệp vào devices của host mà không cần chạy ở chế độ đặc quyền.

- Đạt tiêu chuẩn CIS khi không có container nào được gán thiết bị từ host, có nghĩa là `Devices = []` hoặc `<no value>`.
- Không đạt tiêu chuẩn CIS khi có ít nhất một container được ánh xạ devices từ host, ví dụ như devices có giá trị cụ thể như `/dev/sda`, `/dev/nvidia*`, v.v.).

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: Devices={{.HostConfig.Devices}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Devices=<no value>
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Devices=<no value>
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Devices=<no value>
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: Devices=<no value>
```

Kết quả cho thấy tất cả các container đều có `Devices=<no value>`, tức là không container nào được cấp quyền truy cập trực tiếp vào thiết bị của host.

Đảm bảo ghi đè giới hạn tài nguyên ulimit mặc định tại runtime khi cần thiết

Lý do thực hiện đánh giá tiêu chuẩn này: `ulimit` là cơ chế giới hạn tài nguyên của hệ thống đối với tiến trình, giúp kiểm soát số lượng file mở, tiến trình con và các tài nguyên hệ thống khác. Nếu không cấu hình hợp lý, một container có thể tiêu thụ quá mức tài nguyên, dẫn đến DoS hoặc làm suy giảm hiệu năng toàn hệ thống.

- Đạt tiêu chuẩn CIS khi các container không ghi đè `ulimit` mặc định `Ulimits=<no value>`, hoặc chỉ có cấu hình override trong những trường hợp đặc biệt và có kiểm soát rõ ràng.
- Không đạt tiêu chuẩn CIS khi container tùy ý đặt hoặc thay đổi `ulimit` không hợp lý, không kiểm soát hoặc áp dụng tràn lan.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: Ulimits={{.HostConfig.Ulimits}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Ulimits=[map[Hard:200 Name:nofile Soft:100] map[Hard:2048 Name:nproc Soft:1024]]
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Ulimits=[map[Hard:2048 Name:nproc Soft:1024] map[Hard:200 Name:nofile Soft:100]]
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Ulimits=[map[Hard:2048 Name:nproc Soft:1024] map[Hard:200 Name:nofile Soft:100]]
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: Ulimits=[map[Hard:2048 Name:nproc Soft:1024] map[Hard:200 Name:nofile Soft:100]]
```

Kết quả cho thấy tất cả các container đều có cấu hình Ulimits được thiết lập cụ thể.

Đảm bảo chế độ mount propagation không được đặt là shared

Lý do thực hiện đánh giá tiêu chuẩn này: Mount propagation xác định cách các thay đổi trên filesystem được lan truyền giữa host và container hoặc giữa các mount points. Nếu sử dụng chế độ shared, các thay đổi dữ liệu tại một mount point có thể tự động lan sang các mount point khác, làm giảm tính cô lập giữa các container và giữa container với host.

- Đạt tiêu chuẩn CIS khi tất cả các mount trong container đều sử dụng chế độ private hoặc rprivate (không phải shared).
- Không đạt tiêu chuẩn CIS khi có ít nhất một mount sử dụng propagation mode=shared.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: Propagation={{range $mnt := .Mounts}} {{json $mnt.Propagation}} {{end}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Propagation= "rprivate"
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Propagation= "rprivate"
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Propagation= "rprivate"
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: Propagation=
```

Kết quả cho thấy các container chủ yếu sử dụng propagation="rprivate", không có dấu hiệu sử dụng chế độ shared trong các container được liệt kê, do đó đạt tiêu chuẩn CIS trong mục này và đảm bảo mức độ cô lập filesystem phù hợp.

Đảm bảo không chia sẻ UTS namespace của host với container

Lý do thực hiện đánh giá tiêu chuẩn này: UTS namespace chịu trách nhiệm cô lập các thông tin hệ thống như hostname và domain name giữa host và container. Nếu container được chia sẻ UTS namespace với host, nó có thể thay đổi hostname của hệ thống hoặc truy cập thông tin định danh hệ thống, gây ảnh hưởng đến tính toàn vẹn và an toàn của môi trường vận hành.

- Đạt tiêu chuẩn CIS khi không có container nào sử dụng UTSMode=host.
- Không đạt tiêu chuẩn CIS khi có ít nhất một container sử dụng UTSMode=host.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: UTMMode={{.HostConfig.UTMMode}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: UTMMode=
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: UTMMode=
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: UTMMode=
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: UTMMode=
```

Kết quả cho thấy tất cả các container đều có UTMMode để trống, nghĩa là không chia sẻ UTS namespace với host và đang sử dụng cơ chế cô lập mặc định của Docker.

Đảm bảo seccomp profile mặc định không bị vô hiệu hóa và Đảm bảo container bị hạn chế khỏi việc có thêm đặc quyền bổ sung

Lý do thực hiện đánh giá tiêu chuẩn này: Tiêu chuẩn CIS 5.22 được thực hiện nhằm đảm bảo cơ chế lọc system call của seccomp không bị vô hiệu hóa trên container Docker. Seccomp hoạt động theo cơ chế whitelist, chỉ cho phép các system call cần thiết và chặn các lời gọi khác nhằm giảm bề mặt tấn công của kernel. Việc giữ nguyên hoặc sử dụng seccomp profile tùy chỉnh giúp hạn chế khả năng khai thác các system call nguy hiểm, từ đó tăng cường bảo mật cho container. Trong khi tiêu chuẩn CIS 5.26 được thực hiện nhằm ngăn container có thể tự nâng quyền thông qua các cơ chế như suid hoặc sgid. Tùy chọn no-new-privileges giúp đảm bảo tiến trình bên trong container và các tiến trình con của nó không thể nhận thêm đặc quyền ngoài những quyền đã được cấp ban đầu, giảm nguy cơ leo thang đặc quyền và hạn chế khả năng khai thác các binary đặc quyền trong hệ thống.

- Đạt tiêu chuẩn CIS khi Đối với CIS 5.22, container được xem là đạt chuẩn khi kết quả kiểm tra SecurityOpt không chứa giá trị [seccomp:unconfined].
- Đối với CIS 5.26, container được xem là đạt chuẩn khi trong SecurityOpt xuất hiện tùy chọn no-new-privileges:true.
- Không đạt tiêu chuẩn CIS khi Đối với CIS 5.22, container không đạt chuẩn nếu kết quả kiểm tra trả về [seccomp:unconfined].
- Đối với CIS 5.26, container không đạt chuẩn khi không có tùy chọn no-new-privileges:true trong SecurityOpt, đồng nghĩa với việc tiến trình trong container vẫn có khả năng đạt được thêm đặc quyền thông qua các cơ chế như suid hoặc sgid.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{.Id}}: SecurityOpt={{.HostConfig.SecurityOpt}}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: SecurityOpt=[no-new-privileges:true]
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: SecurityOpt=[no-new-privileges:true]
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: SecurityOpt=[no-new-privileges:true]
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: SecurityOpt=[no-new-privileges:true]
```

Kết quả kiểm tra cho thấy toàn bộ các container hiện tại đều đã được cấu hình với tùy chọn `no-new-privileges:true`, chứng tỏ hệ thống đang tuân thủ tiêu chuẩn CIS 5.26. Về việc ngăn container có thêm đặc quyền bổ sung. Ngoài ra, kết quả cũng không xuất hiện giá trị `[seccomp:unconfined]`, cho thấy seccomp profile mặc định chưa bị vô hiệu hóa và các container vẫn đang được bảo vệ bởi cơ chế lọc system call phù hợp với yêu cầu của CIS 5.22.

Đảm bảo các lệnh docker exec không được sử dụng với tùy chọn đặc quyền (--privileged)

Lý do thực hiện đánh giá tiêu chuẩn này: Tiêu chuẩn CIS 5.23 được thực hiện nhằm đảm bảo rằng các lệnh docker exec không được chạy với tùy chọn `--privileged`, vì tùy chọn này cấp thêm nhiều khả năng và đặc quyền Linux mở rộng cho tiến trình được thực thi bên trong container. Việc sử dụng `--privileged` có thể làm suy giảm các cơ chế bảo vệ đã được cấu hình trước đó, đặc biệt trong các container đang bị giới hạn capability hoặc áp dụng các chính sách bảo mật nâng cao. CIS khuyến nghị chỉ cấp các quyền cần thiết một cách cụ thể thay vì sử dụng toàn quyền thông qua chế độ privileged nhằm giảm nguy cơ leo thang đặc quyền và khai thác hệ thống.

- Đạt tiêu chuẩn CIS khi không phát hiện bất kỳ lệnh docker exec nào được thực thi với tùy chọn `--privileged`.
- Không đạt tiêu chuẩn CIS nếu kết quả từ lệnh kiểm tra `ausearch -k docker | grep exec | grep privileged` trả về các bản ghi liên quan đến việc sử dụng docker exec `--privileged`.

```
# 5.23
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ sudo auditctl -w /usr/bin/docker -k docker
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ sudo auditctl -l | grep /usr/bin/docker
-w /usr/bin/docker -p rwx -k docker
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ sudo ausearch -k docker | grep exec | grep privileged
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$
```

Kết quả kiểm tra cho thấy không phát hiện bất kỳ lệnh docker exec nào sử dụng tùy chọn `--privileged`.

Đảm bảo các lệnh docker exec không được sử dụng với tùy chọn --user=root.

Lý do thực hiện đánh giá tiêu chuẩn này: Tiêu chuẩn CIS 5.24 được thực hiện nhằm đảm bảo rằng các lệnh docker exec không được chạy với tùy chọn `--user=root`, vì tùy chọn này cho phép thực thi lệnh bên trong container với quyền người dùng root, gây mất an toàn, đặc biệt trong các container được thiết kế để chạy dưới tài khoản không đặc quyền như tomcat hoặc các non-root user khác nhằm giảm thiểu rủi ro bảo mật.

- Đạt tiêu chuẩn CIS khi không phát hiện bất kỳ lệnh docker exec nào được thực thi với tùy chọn `--user=root`, đồng thời lệnh kiểm tra `ausearch -k docker | grep`

exec | grep user không trả về kết quả nào liên quan đến việc sử dụng quyền root trong quá trình thực thi lệnh bên trong container.

- Không đạt tiêu chuẩn CIS khi kết quả từ lệnh `ausearch -k docker | grep exec | grep user` trả về các bản ghi cho thấy có sử dụng `docker exec --user=root`.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ sudo ausearch -k docker | grep exec ]
| grep user
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ █
```

Kết quả kiểm tra cho thấy lệnh `ausearch -k docker | grep exec | grep user` không trả về bất kỳ kết quả nào, chứng tỏ không phát hiện việc sử dụng `docker exec --user=root` trên hệ thống.

Đảm bảo việc sử dụng cgroup được xác nhận

Lý do thực hiện đánh giá tiêu chuẩn này: Cgroup là cơ chế quản lý và giới hạn tài nguyên hệ thống cho tiến trình, giúp kiểm soát CPU, bộ nhớ và các tài nguyên khác mà container có thể sử dụng. Nếu container được gắn vào một cgroup khác với cấu hình ban đầu do quản trị viên thiết lập, container có thể nhận thêm quyền hoặc tài nguyên vượt mức cho phép, từ đó làm gia tăng rủi ro bảo mật và ảnh hưởng đến tính cô lập của hệ thống.

- Đạt tiêu chuẩn CIS khi kết quả từ lệnh `docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: CgroupParent={{ .HostConfig.CgroupParent }}'` trả về giá trị `CgroupParent=` để trống cho các container, điều này cho thấy các container đang sử dụng docker cgroup mặc định và không bị gắn vào các cgroup tùy chỉnh ngoài kiểm soát.
- Không đạt tiêu chuẩn CIS khi kết quả kiểm tra trả về một giá trị khác rỗng trong trường `CgroupParent`.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs dock]
er inspect --format '{{ .Id }}: CgroupParent={{ .HostConfig.CgroupParent }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: CgroupParent=
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: CgroupParent=
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: CgroupParent=
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: CgroupParent=
```

Kết quả kiểm tra cho thấy tất cả các container đều có giá trị `CgroupParent=` để trống, chứng tỏ chúng đang chạy dưới docker cgroup mặc định và không có dấu hiệu sử dụng cgroup tùy chỉnh.

Đảm bảo container health được kiểm tra trong runtime.

Lý do thực hiện đánh giá tiêu chuẩn này: Tiêu chuẩn CIS 5.27 được thực hiện nhằm đảm bảo các container đang hoạt động đều được theo dõi health trong runtime thông qua cơ chế `HEALTHCHECK` hoặc tham số `--health-cmd`. Việc kiểm tra sức khỏe container giúp hệ thống phát hiện sớm các trạng thái lỗi, treo dịch vụ hoặc container

hoạt động không đúng mong đợi để có thể thực hiện các biện pháp xử lý kịp thời như restart hoặc thay thế container, nâng cao tính sẵn sàng, ổn định và khả năng giám sát của môi trường Docker.

- Đạt tiêu chuẩn CIS khi tất cả container trả về trạng thái sức khỏe hợp lệ thông qua lệnh `docker ps --quiet | xargs docker inspect --format '{{ .Id }}: Health={{ .State.Health.Status }}'`, đồng thời các container hiển thị trạng thái như healthy cho thấy cơ chế kiểm tra sức khỏe đã được cấu hình và hoạt động đúng.
- Không đạt tiêu chuẩn CIS khi container không có thông tin Health, trả về giá trị rỗng, unhealthy, hoặc không được cấu hình HEALTHCHECK hay `--health-cmd`.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet | xargs docker inspect --format '{{ .Id }}: Health={{ .State.Health.Status }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: Health=healthy
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: Health=healthy
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: Health=healthy
016c429f816c2944fbae96eef02e494d18cd4e4d9d39b4bf30492e5dea21643a: Health=healthy
```

Kết quả kiểm tra cho thấy tất cả các container đều trả về trạng thái Health=healthy, chứng tỏ các container đã được cấu hình cơ chế health check và hiện đang hoạt động ổn định.

Đảm bảo giới hạn PIDs cgroup được sử dụng

Lý do thực hiện đánh giá tiêu chuẩn này: Việc giới hạn PIDs giúp ngăn chặn các cuộc tấn công dạng fork bomb, trong đó tiến trình liên tục sinh thêm tiến trình con làm cạn kiệt tài nguyên hệ thống và có thể khiến host bị treo hoặc phải khởi động lại.

Đạt tiêu chuẩn CIS khi kết quả từ lệnh `docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: PidsLimit={{ .HostConfig.PidsLimit }}'` trả về giá trị PidsLimit khác 0 và -1.

Không đạt tiêu chuẩn CIS khi PidsLimit được đặt là 0 hoặc -1,

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: PidsLimit={{ .HostConfig.PidsLimit }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: PidsLimit=<no value>
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: PidsLimit=<no value>
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: PidsLimit=<no value>
016c429f816c2944fbae96eef02e494d18cd4e4d9d39b4bf30492e5dea21643a: PidsLimit=<no value>
```

Kết quả kiểm tra cho thấy tất cả container đều có giá trị PidsLimit=100, chứng tỏ giới hạn số lượng tiến trình đã được cấu hình và hệ thống đang tuân thủ tiêu chuẩn CIS 5.29.

Đảm bảo không chia sẻ user namespace của host với container

Lý do thực hiện đánh giá tiêu chuẩn này: User namespace giúp ánh xạ người dùng root bên trong container thành người dùng không đặc quyền trên host, từ đó tăng cường khả năng cô lập giữa container và hệ thống. Nếu container chia sẻ user namespace với host,

tiến trình trong container có thể ảnh hưởng trực tiếp đến người dùng trên host và làm suy giảm tính cô lập bảo mật.

- Đạt tiêu chuẩn CIS khi không có container nào sử dụng UsersMode=host, tức là trường UsersMode để trống hoặc sử dụng namespace riêng biệt.
- Không đạt tiêu chuẩn CIS khi có ít nhất một container trả về UsersMode=host.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: UsersMode={{ .HostConfig.UsersMode }}'
e088f1637ab9415cd852abbb571067ddd51c41ea1f98506949815abf35f3f7: UsersMode=
77deb44651ea0d6564de5be9b6ec18ada8d3a7f939bf7878f5698a06e07e202f: UsersMode=
34b641e3905e292a6d4692d839bc2d091f552480fb6a83590eb585f5313f1af6: UsersMode=
016c429f816c2944fbae96eef02e494d18cdae4d9d39b4bf30492e5dea21643a: UsersMode=
```

Kết quả cho thấy tất cả các container đều có giá trị UsersMode= để trống, chứng tỏ không container nào chia sẻ user namespace với host và hệ thống đang tuân thủ tiêu chuẩn CIS 5.31.

Đảm bảo Docker socket không được mount bên trong container

Lý do thực hiện đánh giá tiêu chuẩn này: Docker socket docker.sock cho phép thực thi các lệnh Docker trực tiếp trên host. Nếu socket này được mount vào container, tiến trình bên trong container có thể kiểm soát Docker daemon và gián tiếp giành toàn quyền trên host, gây ra rủi ro bảo mật nghiêm trọng.

Đạt tiêu chuẩn CIS khi lệnh `docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: Volumes={{ .Mounts }}' | grep docker.sock` không trả về kết quả nào, tức là không có container nào mount docker.sock.

Không đạt tiêu chuẩn CIS khi có ít nhất một container mount docker.sock dưới dạng volume.

```
[an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$ docker ps --quiet --all | xargs docker inspect --format '{{ .Id }}: Volumes={{ .Mounts }}' | grep docker.sock
an@an:~/An-CIS-170526/NT542.Q22-Group1-CIS-Docker$
```

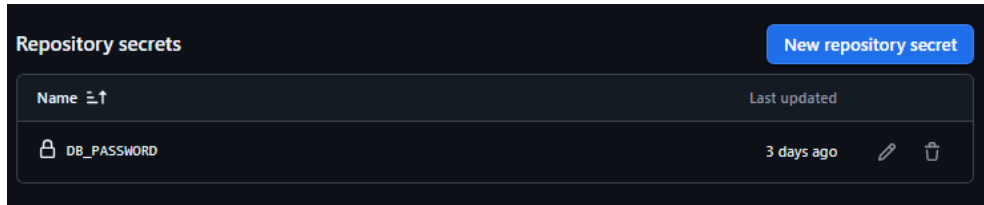
Kết quả cho thấy lệnh kiểm tra không trả về kết quả nào, chứng tỏ không có container nào mount Docker socket và hệ thống đang tuân thủ tiêu chuẩn CIS 5.32.

3. Triển khai Automation

Hệ thống automation được triển khai trực tiếp trên EC2 thông qua self-hosted GitHub Actions Runner. Toàn bộ quy trình CI/CD được cấu hình trong file workflow `.github/workflows/deploy.yml`, kết hợp với Docker Compose và script kiểm tra CIS Benchmark nhằm tự động hóa việc build, deploy và audit hệ thống container.

3.1. Cấu hình GitHub Secrets

Thông tin nhạy cảm như mật khẩu database không được hardcode mà được quản lý thông qua GitHub Secrets. Trong quá trình chạy workflow, hệ thống tự động tạo file `.env` trên EC2 để Docker Compose sử dụng.



File `.env` được inject vào container MySQL và application service thông qua Docker Compose environment variable mapping, đảm bảo tách biệt cấu hình và source code.

3.2. Workflow CI/CD trên GitHub Actions

Pipeline triển khai được định nghĩa trong file `.github/workflows/deploy.yml` với các bước chính:

Bước 1: Checkout source code từ repository

```
- uses: actions/checkout@v4
```

Bước 2: Tạo file `.env` từ GitHub Secrets

```
- name: Create .env
  run: |
    echo "DB_PASSWORD=${{ secrets.DB_PASSWORD }}" > .env
```

Bước 3: Build lại Docker images bằng Docker Compose

```
- name: Deploy Containers
  run: docker compose up -d --build
```

Bước 4: Kiểm tra trạng thái container

```
- name: Check Containers
  run: docker ps
```

Bước 5: Chạy CIS Docker Benchmark Script

```
- name: Run CIS Audit
  run: bash audit.sh
```

3.3. Thiết kế script CIS Audit

Hệ thống sử dụng script Bash audit/run_all.sh để tự động đánh giá mức độ tuân thủ CIS Docker Benchmark. Script được thiết kế theo mô-đun, mỗi module kiểm tra một nhóm tiêu chí bảo mật.

Cấu trúc script:

- Module 1: Kiểm tra Docker daemon và runtime
- Module 2: Kiểm tra container security configuration
- Module 3: Kiểm tra image security (USER, ADD, secrets, base image)
- Module 4: Kiểm tra host-level security và file permission

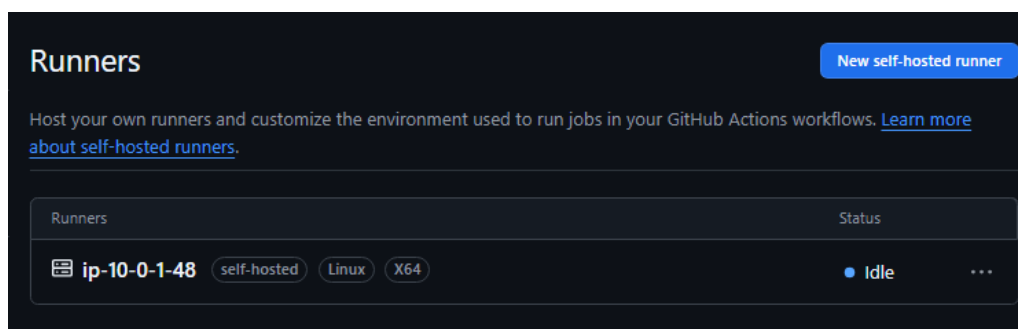
Luồng hoạt động của script:

- 1) Thu thập danh sách images và containers từ Docker engine
- 2) Inspect cấu hình container (user, privilege, security options)
- 3) Phân tích Docker image history để phát hiện rủi ro (secret, apt-get, ADD)
- 4) Kiểm tra runtime container (SUID, package tools, updates)
- 5) Ghi nhận kết quả PASS/FAIL/WARN theo từng tiêu chí CIS
- 6) Tổng hợp báo cáo hiển thị trên GitHub Actions log hoặc xuất file report

Script hoạt động hoàn toàn tự động sau mỗi lần deployment nhằm đảm bảo trạng thái hệ thống luôn được kiểm tra theo chuẩn CIS.

3.4. Liên kết EC2 với GitHub Actions (kèm cơ chế API & authentication)

Trong quá trình thiết lập CI/CD, EC2 được liên kết với GitHub thông qua cơ chế Self-hosted Runner. Việc đăng ký runner không chỉ là chạy script cài đặt mà còn sử dụng GitHub API token để xác thực và gắn máy chủ EC2 vào repository.



Cơ chế API khi cài đặt GitHub Runner

Khi cấu hình runner, hệ thống yêu cầu một registration token được tạo từ GitHub repository thông qua GitHub REST API. Token này có thời gian sống ngắn và chỉ dùng cho bước đăng ký ban đầu.

Quy trình diễn ra như sau:

- GitHub cung cấp registration token thông qua API (hoặc UI repository settings)
- EC2 sử dụng token này trong lệnh `./config.sh`
- Runner gửi request xác thực đến GitHub API
- GitHub ghi nhận EC2 như một self-hosted runner hợp lệ
- Sau khi xác thực thành công, runner được liên kết cố định với repository

Lệnh cài đặt và đăng ký runner (có API token)

```
mkdir actions-runner
cd actions-runner

curl -o actions-runner.tar.gz -L <download-url>
tar xzf actions-runner.tar.gz

./config.sh --url https://github.com/<repo> \
            --token <RUNNER_REGISTRATION_TOKEN>
```

Trong đó:

- `--url`: endpoint repository GitHub
- `--token`: token được sinh từ GitHub API (Token này chỉ dùng để đăng ký runner, không dùng trong runtime CI/CD).

Sau khi runner được đăng ký thành công:

- GitHub Actions gửi job event đến GitHub API
- GitHub dispatch job xuống EC2 thông qua runner agent
- Runner trên EC2 poll GitHub API liên tục để nhận job
- Khi có job, runner tải workflow và thực thi trực tiếp trên EC2

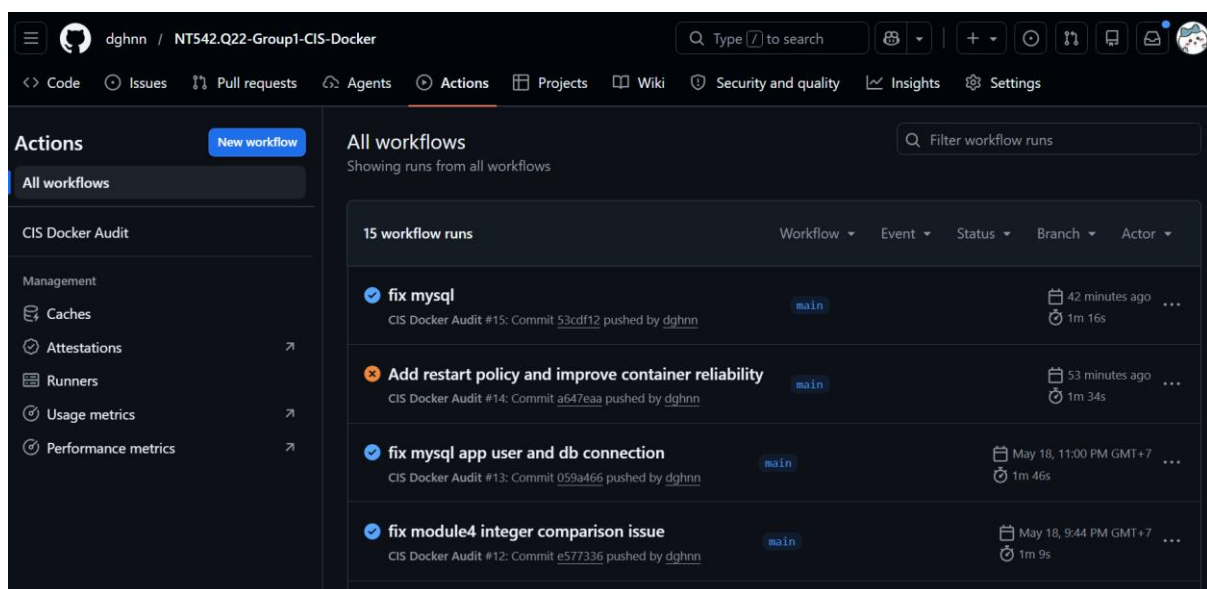
Chương IV. DEMO

Demo quy trình Automation

Để kiểm chứng cơ chế tự động hóa, một thay đổi nhỏ được thực hiện trên mã nguồn và đẩy lên GitHub Repository. Sau khi lệnh push được thực hiện, GitHub Actions tự động kích hoạt workflow CI/CD đã được cấu hình trước.

Self-hosted Runner trên EC2 tiếp nhận workflow và thực hiện tuần tự các tác vụ bao gồm tải mã nguồn mới nhất từ GitHub, tạo file cấu hình môi trường từ GitHub Secrets, build lại Docker image, khởi động hoặc cập nhật các container bằng Docker Compose và kiểm tra trạng thái hoạt động của hệ thống.

Toàn bộ quá trình được thực hiện tự động mà không cần thao tác trực tiếp trên máy chủ. Kết quả thực thi được hiển thị trên giao diện GitHub Actions dưới dạng workflow log, cho phép theo dõi chi tiết từng bước triển khai.



Workflow được kích hoạt tự động sau khi source code được cập nhật lên GitHub.

CIS Docker Benchmark Audit

Sau khi hệ thống được triển khai thành công, workflow tiếp tục thực hiện bước đánh giá bảo mật dựa trên CIS Docker Benchmark. Script `run_all.sh` được gọi để thực hiện lần lượt các module kiểm tra đã xây dựng.

Quá trình audit bao gồm kiểm tra cấu hình host, Docker daemon, Docker image và môi trường runtime của container. Kết quả của từng tiêu chí được phân loại thành PASS, FAIL hoặc WARN và được ghi nhận vào báo cáo tổng hợp dưới định dạng CSV.

Bên cạnh việc hiển thị kết quả trực tiếp trên workflow log, báo cáo audit còn được tự động đóng gói và lưu trữ dưới dạng Artifact trên GitHub Actions để phục vụ việc phân tích và đối chiếu sau này.

```

Show runner info 0s
1 ▶ Run hostname
6 ip-10-0-1-48
7 ubuntu
8 Docker version 29.1.3, build 29.1.3-0ubuntu3-24.04.2

Create .env 0s
1 ▶ Run echo "DB_PASSWORD=****" > .env

Start Docker Lab 6s
1 ▶ Run docker compose up -d --build
4 #0 building with "default" instance using docker driver
5
6 #1 [app internal] load build definition from Dockerfile
7 #1 transferring dockerfile: 781B 0.0s done
8 #1 DONE 0.1s
9
10 #2 [app internal] load metadata for docker.io/library/php:8.2-fpm
11 #2 DONE 0.1s
12
13 #3 [app internal] load .dockerignore
14 #3 transferring context: 2B done
15 #3 DONE 0.0s
16
17 #4 [app 1/7] FROM docker.io/library/php:8.2-fpm@sha256:61f68255ebab17fa34822c6130ba98f392418eebf4fece1856f8d2782bfd3076
18 #4 resolve docker.io/library/php:8.2-fpm@sha256:61f68255ebab17fa34822c6130ba98f392418eebf4fece1856f8d2782bfd3076 0.0s done
19 #4 DONE 0.0s
20

```

```

Show containers 0s
1 ▶ Run docker ps
4
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS                    PORTS
5 08c3c7de72dd  nt542q22-group1-cis-docker-nginx   "/docker-entrypoint..." 2 seconds ago  Up Less than a second (health: starting) 80/tcp, 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
6 14fef1cd49f9  nt542q22-group1-cis-docker-nginx-1 "/docker-php-entrypoi..." 3 seconds ago  Up Less than a second (health: starting) 9000/tcp
7 3d1e57eb1de7  redis:alpine                       "docker-entrypoint.s..." 11 minutes ago  Up 9 minutes (healthy) 6379/tcp
8 4ace078f73c0  mysql:8                             "docker-entrypoint.s..." 11 minutes ago  Up 9 minutes (healthy) 3306/tcp, 33060/tcp
9 nt542q22-group1-cis-docker-db-1

Run CIS audit 59s
1 ▶ Run chmod +x audit/run_all.sh
5 ===== CIS DOCKER FULL AUDIT =====
6 [RUN] module1
7 [RUN] module2
8 [RUN] module3
9 [RUN] module4
10
11 ===== DONE =====
12 CSV REPORT: report/cis_report.csv

```

Kết quả thực thi CIS Audit trên GitHub Actions và báo cáo được sinh tự động sau khi hoàn tất kiểm tra.

```

Upload CSV Report 1s
1 ▶ Run actions/upload-artifact@v4
9 With the provided path, there will be 1 file uploaded
10 Artifact name is valid!
11 Root directory input is valid!
12 Beginning upload of artifact content to blob storage
13 Uploaded bytes 2970
14 Finished uploading artifact content to blob storage!
15 SHA256 digest of uploaded artifact zip is 70e236d0f59027b095121e400228c372aa0e7281978f6abfaac6d6b6b3f20f5b
16 Finalizing artifact upload
17 Artifact cis-csv-report.zip successfully finalized. Artifact ID 7067202972
18 Artifact cis-csv-report has been successfully uploaded! Final size is 2970 bytes. Artifact ID is 7067202972
19 Artifact download URL: https://github.com/dghnn/NT542-Q22-Group1-CIS-Docker/actions/runs/26855430112/artifacts/7067202972

```

Phân tích Kết luận

Báo cáo thực hiện đánh giá bảo mật môi trường Docker dựa trên tiêu chuẩn CIS Docker Benchmark với 4 nhóm kiểm tra chính:

- Module 1: Host Configuration
- Module 2: Docker Daemon Configuration
- Module 3: Container Images & Build File
- Module 4: Container Runtime Security

Kết quả kiểm tra cho thấy hệ thống đã triển khai một số cơ chế bảo mật nền tảng tương đối tốt, đặc biệt ở các nội dung liên quan đến phân quyền file cấu hình, cô lập container và hạn chế leo thang đặc quyền. Tuy nhiên, hệ thống vẫn tồn tại nhiều rủi ro bảo mật quan trọng cần được khắc phục, đặc biệt ở các vấn đề liên quan đến auditing, logging tập trung và giới hạn tài nguyên container.

2.1. Module 1 – Host Configuration

Ưu điểm:

- Các file và thư mục cấu hình quan trọng của Docker như:

đều được cấu hình owner và permission đúng theo khuyến nghị CIS.

- Firewall trên hệ thống đang hoạt động và hỗ trợ hạn chế truy cập trái phép từ mạng.
- Nhược điểm:
- Thư mục `/var/lib/docker` chưa được mount trên một phân vùng độc lập, dẫn đến nguy cơ Docker sử dụng toàn bộ dung lượng ổ đĩa hệ điều hành.
- SSH root login chưa được hardening đầy đủ, vẫn tiềm ẩn nguy cơ brute-force hoặc truy cập trái phép.

2.2. Module 2 – Docker Daemon Configuration

Ưu điểm:

Docker daemon không mở cổng TCP từ xa, giúp giảm nguy cơ bị tấn công trực tiếp qua mạng.

Nhược điểm:

Thiếu cơ chế Auditing

- Hệ thống chưa cài đặt auditd.
- Các thành phần quan trọng như:
 - + dockerd
 - + containerd
 - + docker.sock
 - + runc

chưa được giám sát bằng audit rule.

Điều này khiến quản trị viên không thể phát hiện hoặc truy vết các thay đổi trái phép liên quan đến Docker daemon.

Docker Daemon chạy bằng quyền root

- Docker daemon hiện vẫn hoạt động ở chế độ rootful thay vì rootless.
- Nếu daemon bị khai thác, attacker có thể chiếm quyền điều khiển toàn bộ host.

Logging chưa tối ưu

- Hệ thống vẫn sử dụng local logging (json-file) thay vì centralized logging.
- Việc này gây khó khăn cho quá trình điều tra và phân tích sự cố bảo mật.

2.3. Module 3 – Container Images & Build Security

Ưu điểm:

Các image ứng dụng tự build đã áp dụng khá tốt các cơ chế hardening:

- Sử dụng USER non-root
- Thiết lập HEALTHCHECK
- Bật no-new-privileges
- Sử dụng seccomp

Các cấu hình này giúp giảm nguy cơ privilege escalation trong container runtime.

Nhược điểm:

Base image chưa an toàn

Các base image như:

- nginx:alpine
- mysql:8
- php:8.2-fpm
- redis:alpine

vẫn chạy bằng root và chưa cấu hình HEALTHCHECK.

Attack Surface lớn

- Một số image chứa nhiều package và development tools không cần thiết.
- Tồn tại nhiều file có SUID/SGID bit, làm tăng nguy cơ khai thác.

Dockerfile chưa tối ưu

- Lạm dụng lệnh ADD thay vì COPY.
- Chưa bật Docker Content Trust.
- Có nguy cơ lộ secrets thông qua Image History.

2.4. Module 4 – Container Runtime Security

Ưu điểm:

Môi trường runtime được cô lập tương đối tốt:

- Không sử dụng host network
- Không chia sẻ PID namespace
- Không mount các thư mục nhạy cảm từ host
- Có cấu hình no-new-privileges
- Namespace và device isolation hoạt động đúng

Nhược điểm:

Không giới hạn tài nguyên

Tất cả container hiện chưa giới hạn:

- CPU
- RAM

Điều này có thể dẫn đến:

- Resource exhaustion
- Denial of Service (DoS)
- Treo toàn bộ host nếu container gặp lỗi

Restart Policy chưa phù hợp

- Đang sử dụng:

restart: unless-stopped

- Điều này có thể gây vòng lặp restart vô hạn nếu container lỗi liên tục.

Cấu hình mạng chưa tối ưu

- Nginx đang bind port ra 0.0.0.0, làm tăng phạm vi truy cập từ mạng ngoài.

Sử dụng tag latest

- Container nginx và app vẫn sử dụng tag latest.
- Điều này đi ngược best-practice vì có nguy cơ:
 - + Cập nhật ngoài ý muốn
 - + Lỗi tương thích
 - + Sử dụng image giả mạo

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên	Nội dung thực hiện	Mức độ hoàn thành
Bùi Nguyễn Tường Vân	Thực hiện kiểm tra và đánh giá CIS Docker Module 1; Thiết kế kiến trúc hệ thống	100%
Vũ Duy An	Thực hiện CIS Docker Module 2; cài đặt Docker, Docker Compose, cấu hình EC2 và Security Group	100%
Đỗ Gia Hân	Thực hiện CIS Docker Module 3; xây dựng GitHub Actions, GitHub Runner, script automation audit	100%
Phan Nguyễn Hoài An	Thực hiện CIS Docker Module 4; kết quả audit và thực hiện demo hệ thống thực tế	100%

NGUỒN THAM KHẢO

- [1] Center for Internet Security (CIS), *CIS Benchmarks Documentation*, Available: <https://www.cisecurity.org/cis-benchmarks>.
- [2] Docker Inc., *Docker Documentation – Security*, Available: <https://docs.docker.com/engine/security/>.
- [3] Docker Inc., *Docker Compose Documentation*, Available: <https://docs.docker.com/compose/>.
- [4] GitHub, *GitHub Actions Documentation*, Available: <https://docs.github.com/en/actions>.
- [5] Amazon Web Services (AWS), *Amazon EC2 Documentation*, Available: <https://docs.aws.amazon.com/ec2/>.